Going Live

Deploying the Copper–Gas–Bitcoin–Sol Recycling Portfolio on Oracle Cloud

A complete runbook

Robin Langer · August 2026

This document is written to be executed. Claude reads it and does the engineering. Kim does the four things that legally and physically cannot be delegated: proving who he is, funding the accounts, and authorising the keys.

Contents

I Orientation **4**

1 Who Does What **4**

2 What Already Exists **4**

3 Where It Runs, and the SSH Question **5**
- 3.1 No, you do not hand over your SSH credentials . . . 5
- 3.2 How it actually works . . . 5
- 3.3 Kim's side: generate a key . . . 6
- 3.4 Robin's side: add Kim, in one command . . . 6
- 3.5 The trust boundary, stated honestly . . . 6
- 3.6 Kim's laptop: fetching reports . . . 7

4 Step 2 — Crypto Exchange for Bitcoin and Solana **7**

5 Step 3 — Broker for Copper and Natural Gas **8**
- 5.1 The alternative, and its honest cost . . . 9
- 5.2 Which broker . . . 9

6 Step 4 — Funding, and Your Banking Details **10**

II Wallets, Accounts, and Keys **11**

7 Wallet Versus Exchange Account **11**
- 7.1 So do we need a wallet at all? . . . 11

8 Where Keys Live on the Server **12**

III Claude's Work **12**

9 SSH: Getting In **12**

10 Installing the Scripts **13**

11 The Cron Job **14**

12 The Heartbeat: Not Being Reclaimed **14**
- 12.1 How the keepalive works . . . 15
- 12.2 The other heartbeat: knowing it is alive . . . 15

13 The Signal Engine **16**
- 13.1 Two deliberate design decisions . . . 16

14 The Execution Layer (Not Yet Written) **16**

IV The Weekly Report **17**

15 What It Produces **17**

16 The Design Decision That Matters **18**

17 How Claude Fetches It **18**

V Going Live **18**

18 Choosing the Solana Cap **18**
18.1 Why 25%, given the plateau 19

19 Four Phases, With Gates **20**

20 What Would Stop Us **21**

21 Command Reference **21**

VI Appendix: The Complete Code **23**

22 signals.py **23**

23 weekly_report.py **26**

24 cap_sweep.py **30**

25 run_weekly.sh **32**

26 keepalive.sh **32**

27 healthcheck.sh **33**

28 install_oci.sh **34**

29 fetch_report.sh **35**

30 make_figure.py **35**

Part I

Orientation

1. Who Does What

There are three actors, and the boundaries matter — both for security and because some steps are legally personal to the account holder.

Actor	Responsibilities
Kim	Identity verification (KYC) at each venue; funding accounts from his own bank; generating API keys and choosing their permissions; approving anything that moves money out. *Cannot be delegated.*
Claude	Everything technical: provisioning the server, SSH, installing the scripts, the virtualenv, cron, the signal engine, the execution layer, the weekly reports, monitoring, and debugging.
Robin	Owns the strategy and the research. Reviews the go-live decision at each phase gate. Holds nothing of Kim's.

The security model, stated once and applied throughout

Kim never hands over a bank login, a password, or a seed phrase. Not to Robin, not to Claude, not to anyone. Anybody who asks for those is either mistaken or stealing.

What Kim *does* hand over is a set of **API keys scoped to trading only, with withdrawal permission switched off and locked to the server's IP address**. That is the entire difference between "a system that can rebalance my portfolio" and "a system that can empty my account." An API key without withdrawal rights cannot move money off the venue — the worst a total compromise achieves is bad trades, which is bounded, recoverable, and insurable by position limits.

Kim's banking details are typed by Kim, into the broker's own deposit screen, and go nowhere else. See Section 6.

2. What Already Exists

Not starting from zero. As of August 2026:

- **An OCI instance is already running** — `ubuntu@207.211.145.30`, Ubuntu 24.04.4 LTS on `aarch64` (ARM), Python 3.12.3, 23 GB RAM, 69 GB free disk, `cron` active with ten existing jobs. SSH alias `oci`. It hosts four AI agents in Docker plus the Gateway app.
- **The research is done** — see `algorithm.pdf` (the complete, self-contained code that generates the portfolio comparison), `kim.pdf` (the full cross-asset study) and `concepts.pdf` (the primer for Kim).
- **What does not exist yet** — the execution layer (code that places orders), any funded

account, and a measurement of how much of the copper and gas alpha survives in a tradeable instrument. All three are addressed below.

Note: `~/git/oci` on Robin's Mac is a plain directory, *not* a git repository, and there is no GitHub repo named `oci` under either `RaggedR` or `nickmeinhold`. If you want this versioned, it needs `git init` and a new remote.

3. Where It Runs, and the SSH Question

The reports run on **Robin's existing instance** (`207.211.145.30`). It is already provisioned, already has cron, and costs nothing. Kim needs no cloud account at all.

Kim runs nothing on the server himself. He works from **his own machine**, where he does exactly two things: fetch the weekly report, and — optionally — log in to look around.

3.1 No, you do not hand over your SSH credentials

The question is the right one to ask, and the answer is unambiguous:

Never share a private key. With anyone. For any reason.

An SSH private key is not a password that can be scoped or revoked per-person — it *is* your identity on every machine that trusts it. Sharing `~/.ssh/oci_new` would give Kim Robin's access to Robin's server, indistinguishably from Robin, with no way to tell the two apart in a log and no way to revoke one without revoking both.

That is not how SSH is meant to be used, and it is not necessary. **Public-key authentication is designed for exactly this situation.**

3.2 How it actually works

Keys come in pairs. The **private** half never leaves the machine that will connect. The **public** half is meant to be handed out — that is its entire purpose — and it is useless for anything except recognising its private twin.

So: **Kim generates his own pair on his own laptop and sends Robin only the public half.** Robin adds that line to the server. Now Kim logs in as Kim, with a key Robin has never seen, and Robin's own key is untouched.

Property	Consequence
Robin's key unchanged	Nothing to rotate, nothing exposed.
Kim has his own identity	The logs distinguish them.
Revocation is one line	Delete Kim's line from `authorized_keys` and his access ends instantly, without affecting Robin.
Separate Unix account	Kim's files, keys and reports live under `/home/kim`, not mixed into Robin's home directory.

3.3 Kim's side: generate a key

Step 1: make an SSH key — 5 minutes

1. Open Terminal (macOS/Linux), or PowerShell on Windows 10+.
2. Run, exactly as written:

   `ssh-keygen -t ed25519 -C "kim-portfolio"`

   Press Enter to accept the default location. **Set a passphrase** when asked — if your laptop is ever stolen, that passphrase is the only thing between the thief and the server.
3. Print the **public** half:

   `cat ~/.ssh/id_ed25519.pub`
4. Send Robin **that whole line**. It starts with `ssh-ed25519` and ends with `kim-portfolio`. Email is fine for this one — it is a public key, and it is safe by design.
5. **Never send the other file** (`~/.ssh/id_ed25519`, no `.pub`). That one is yours forever. If anyone ever asks you for it, the answer is no.

3.4 Robin's side: add Kim, in one command

Run once, from Robin's Mac

```
KIMKEY='ssh-ed25519 AAAA... kim-portfolio' # the line Kim sent

ssh oci "sudo bash -s" <<EOF
set -euo pipefail
# A normal login user. Deliberately NOT added to the sudo group.
id kim >/dev/null 2>&1 || sudo adduser --disabled-password --gecos "" kim
install -d -m 700 -o kim -g kim /home/kim/.ssh
touch /home/kim/.ssh/authorized_keys
grep -qF "kim-portfolio" /home/kim/.ssh/authorized_keys \
  || echo "$KIMKEY" >> /home/kim/.ssh/authorized_keys
chmod 600 /home/kim/.ssh/authorized_keys
chown -R kim:kim /home/kim/.ssh
echo "kim added; sudo group membership:"; groups kim
EOF
```

Kim then adds this to his own `/.ssh/config`

```
Host portfolio
  HostName 207.211.145.30
  User kim
  IdentityFile ~/.ssh/id_ed25519
```

and `ssh portfolio` works from Kim's laptop. To revoke, ever:

Revocation

```
ssh oci "sudo sed -i '/kim-portfolio/d' /home/kim/.ssh/authorized_keys"
# or remove the account entirely:
ssh oci "sudo deluser --remove-home kim"
```

3.5 The trust boundary, stated honestly

Two things follow from sharing a machine, and both should be said out loud rather than discovered later.

1. **Robin has root. Robin can read Kim's API keys.** No file permission changes this; that is what root means. The arrangement rests on trust between brothers, which is fine — but it is trust, not a technical guarantee, and Kim should know which one he is relying on.
2. **Kim gets no `sudo`.** Not as a slight: the box also runs four AI agents and the Gateway app, and an accidental `sudo` command from someone unfamiliar with it could take those down.

Kim needs nothing more than read access to his own report directory.

What makes this acceptable

The reason a shared box is tolerable here is the design decision in Section 8: **the API keys have withdrawal disabled.** The worst outcome from any compromise — Robin's, Kim's, or an agent's — is unwanted trades, not stolen funds.

If that ever stops being true, or if the amount grows past the point where "I trust my brother's server" is a sentence Kim is comfortable saying, the answer is a separate Always Free tenancy in Kim's own name. That is an hour of clicking, the scripts are host-portable, and `install_oci.sh` takes an `OCI_HOST` variable specifically so the move is a one-line change.

3.6 Kim's laptop: fetching reports

Step 2: get the weekly report — 2 minutes, once

1. The report is written every Monday morning whether or not anyone looks. Nothing breaks if you ignore it for a month.
2. The numbers, without downloading anything:

   ```
   ssh portfolio 'cat ~/portfolio/reports/latest.json'
   ```

3. The PDF, copied to your Desktop:

   ```
   scp portfolio:~/portfolio/reports/latest.pdf ~/Desktop/
   ```

4. If you would rather not touch a terminal at all, say so — Robin can have the server email or message you the PDF on Monday instead. There is no requirement for Kim to use SSH.

4. Step 2 — Crypto Exchange for Bitcoin and Solana

Why an exchange and not just a wallet: this is the single most common point of confusion, and Section 7 explains it properly. The short version: a *wallet* holds coins; an *exchange account* can buy and sell them. A strategy that trades needs an exchange account with an API. We will use a wallet too, later, as a vault.

Recommended venue: Kraken. Long operating history, serves Australian customers with AUD deposits, lists both Bitcoin and Solana, and — decisively for us — has a straightforward API-key system with *granular permissions* and no requirement to re-authenticate by hand every day. Verify at signup that it still serves your state; if not, the alternatives are Independent Reserve, BTC Markets, or Coinbase Advanced, all of which have comparable APIs.

Step 2: open and verify a Kraken account — 30 minutes plus 1–3 days verification

1. Go to `https://www.kraken.com` and create an account with a **unique password from your password manager**.
2. Turn on two-factor authentication immediately, using your authenticator app. **Not** SMS — SIM-swap attacks on crypto accounts are routine.
3. Complete identity verification: photo ID, a selfie, and your address. Choose the tier that permits AUD deposits and trading. Approval usually takes hours; occasionally days.
4. **Do not deposit anything yet.** Funding happens at Phase 3, after the system has been proven on paper.
5. Now create the API key. **Settings** → **API** → **Add key**, and set permissions *exactly* as follows:

Permission	Setting
Query funds / balances	**ON**
Query open & closed orders	**ON**
Create & modify orders	**ON**
Cancel / close orders	**ON**
Withdraw funds	**OFF — critical**
Deposit funds	**OFF**
Access WebSockets	**ON**
Export data	**ON**

6. If Kraken offers to restrict the key to specific IP addresses, **say yes** and enter the server address Robin gives you. This means a stolen key is useless from anywhere else on earth.
7. Kraken shows the **API Secret exactly once.** Copy both the Key and the Secret straight into your password manager before closing the page.
8. **Send Robin** the API Key and Secret — and send them **through a channel that is not email**: Signal, or a password-manager shared item. Email is stored forever on servers you do not control.

Why "Withdraw funds: OFF" is the most important switch in this document

With withdrawal off, the absolute worst case — server compromised, keys stolen, attacker in full control — is that somebody makes stupid trades inside Kim's account. Unpleasant, bounded, and partly recoverable.

With withdrawal on, the worst case is that the money is gone in ninety seconds and is never coming back. Crypto transactions are irreversible by design; there is no chargeback and no fraud department.

The convenience gained by enabling withdrawal is zero, because nothing in this system ever needs to move money off the exchange.

5. Step 3 — Broker for Copper and Natural Gas

Copper and natural gas are not coins; they are commodities, traded as futures contracts on the Chicago Mercantile Exchange. This leg needs a real broker, and there is a genuine problem to solve first.

The problem: the contracts are far too big

Our backtest used front-month futures prices (`HG=F`, `NG=F`). Buying one of those contracts commits you to a great deal more money than the whole portfolio:

Contract	Size	Approx. notional
Copper (HG)	25,000 lb	~$110,000
Micro copper (MHG)	2,500 lb	~$11,000
Henry Hub natgas (NG)	10,000 MMBtu	~$29,000
E-mini natgas (QG)	2,500 MMBtu	~$7,250

On a $10,000 portfolio, a single copper contract is eleven times the account. Even the micro contract is more than the entire intended copper allocation. Futures are leveraged instruments — you post a margin deposit, not the full value — which means a position that looks affordable can lose more than you deposited. **The backtest assumed no leverage. Trading these contracts directly would not be implementing the strategy; it would be implementing a much riskier strategy that happens to share its signals.**

5.1 The alternative, and its honest cost

The way to get unleveraged, correctly-sized exposure is an exchange-traded fund that tracks the commodity: **CPER** for copper, **UNG** for natural gas. These trade like shares, can be bought in any quantity (fractionally, at some brokers), and cannot lose more than you put in.

The cost of that convenience is real and must be stated:

1. **Roll decay.** These funds hold futures and must repeatedly sell expiring contracts to buy later ones. When later contracts are more expensive — the usual state of affairs in natural gas — this bleeds value continuously. **UNG has historically underperformed the natural gas price it tracks by a wide margin.** Our backtest measured `NG=F` and will therefore *overstate* what UNG delivers. The strategy's saving grace is that it holds gas only about half the time, but the drag is not eliminated, merely reduced.
2. **Tax complexity.** Both are structured as commodity pools and issue US K-1 style tax documents rather than ordinary dividend statements. For an Australian taxpayer this is a genuine accounting nuisance. Ask an accountant before Phase 3.
3. **Not all brokers permit them.** Verify at signup.

Recommendation, stated plainly

Trade the crypto leg for real; keep the copper and gas leg on paper until the instrument question is settled.
The Bitcoin and Solana legs are clean: spot, unleveraged, one venue, a good API. They can go live as soon as Kim is verified and funded.
The copper and gas legs have an unresolved instrument-selection problem — futures are too large, and the ETFs carry decay and tax costs the backtest never measured. Running them on paper for a quarter costs nothing and tells us how much of the backtested alpha actually survives in the tradeable instrument. **That measurement should be made before Kim's money is committed, not after.**

5.2 Which broker

Position: Interactive Brokers. It is the standard answer — access to CME futures, US ETFs, an Australian entity, and a well-documented API that `ib_async` speaks fluently.

Objection. IBKR's API is built around a desktop Java application (Trader Workstation or IB Gateway) that must be running for the API to answer, and its session requires two-factor re-authentication roughly daily. On a headless ARM server this means a Java gateway under a virtual framebuffer, plus Kim tapping "approve" on his phone most days. For a system whose entire selling point is that it runs unattended, a daily human tap is a design flaw, not a detail.

Resolution. Choose the broker by what the automation needs, not by reputation. For the ETF route — which is what we recommend — a broker with plain API-key authentication and no daily re-auth is strictly better. **Alpaca** is the natural fit: pure REST, keys that do not expire daily, fractional shares, a built-in paper-trading environment that is API-identical to live, and support for US-listed ETFs. Keep IBKR in reserve for the day we decide we genuinely want futures exposure and accept the operational burden.

Practical difference: Alpaca can be automated tonight, in paper mode, with no account at all. IBKR cannot be automated unattended without solving the 2FA problem first.

Step 3: open a broker account — 30 minutes plus verification

1. **Wait.** This step is deliberately last and is not needed for Phases 1–2. Do not open it until Robin confirms the instrument decision.
2. When told to proceed, go to `https://alpaca.markets`, create an account, and confirm during signup that Australian residents are accepted and that `CPER` and `UNG` are tradeable. If either answer is no, tell Robin and stop — do not improvise a substitute instrument.
3. Complete identity verification as in Step 2. Enable two-factor authentication.
4. Generate API keys for the **paper** environment first and send those to Robin. Paper keys cannot touch money, so they are safe to hand over early and they let Claude prove the whole pipeline works.
5. Generate **live** keys only when Robin confirms Phase 3. Restrict to trading; no funds-transfer permission; IP-restricted if offered.
6. **Send Robin** the key and secret over Signal, never email. Label clearly which are paper and which are live.

6. Step 4 — Funding, and Your Banking Details

Step 4: funding — only at Phase 3, and only by you

1. Decide the amount, and apply one test: **if this money vanished entirely, would anything in your life break?** If yes, the number is too big. Read the drawdown table in `concepts.pdf` before answering — an 80% temporary loss is inside the historical range of what this strategy does.
2. Log in to the venue **yourself**. Find Deposit. Choose AUD bank transfer (PayID/Osko is usually instant; a normal transfer takes a day).
3. **Type your own bank details into the venue's deposit screen.** Copy the reference code exactly — deposits without it get held up.
4. Transfer from your bank as you would to any payee. The first transfer may be held for review; this is routine.
5. **Tell Robin the amount and the date. Send no account numbers, no BSB, no bank password, no card number.** Nothing in this system needs them, and nobody who does need them would ask you through a person.
6. Start with the smallest amount that makes the exercise real — a few hundred dollars, not the full allocation. Phase 4 is where the size increases, after Phase 3 has run clean.

A note on whose money this is

The accounts are in **Kim's own name**, funded from Kim's own bank, and Kim can close them or revoke the keys at any moment without asking anyone. That is deliberate, and it should stay that way.

It keeps custody with Kim, and it keeps the arrangement clearly on the right side of a real regulatory line: in Australia, managing other people's money as a business generally requires an Australian Financial Services licence. A brother sharing research and infrastructure is not that. Pooled funds, a fee, or discretion over someone else's account would be. If this ever grows past two brothers and a shared spreadsheet, get advice first.

Part II

Wallets, Accounts, and Keys

7. Wallet Versus Exchange Account

Four different things get called "a wallet," and conflating them is how people lose money.

Thing	What it is	Can it trade?
Exchange account	An account at a company (Kraken) that holds coins on your behalf and matches your buy and sell orders.	**Yes** — and it is the only one of these four that can. This is what the strategy uses.
Software wallet	An app holding your private keys (Phantom for Solana, Sparrow for Bitcoin). You control the coins directly.	No. It can send and receive, not trade.
Hardware wallet	A dedicated device (Ledger, Trezor) keeping keys offline. The gold standard for storage.	No.
Paper backup	Twelve or twenty-four words written down. *Is* the wallet, in every sense that matters.	No.

The one rule about seed phrases

A seed phrase **is** the money. Anyone holding those words owns the coins, permanently and irreversibly.

Therefore: write it on paper, never photograph it, never type it into anything except the wallet app that generated it, never put it in email or cloud storage, and **never send it to Robin, to Claude, or to any person or program that asks.** There is no legitimate reason for anyone to see it. Every message that has ever asked for one was a theft in progress.

7.1 So do we need a wallet at all?

For trading, no. For safety, yes — eventually.

Leaving coins on an exchange means trusting the exchange to stay solvent and unhacked, and the history of the industry is unkind on both counts. The rule of thumb: the exchange holds the working capital the strategy is actively trading; a hardware wallet holds anything you have decided to keep.

Optional Step 5: a hardware wallet — do this at Phase 4, not now

1. Buy a Ledger or Trezor **direct from the manufacturer's own website**. Never from eBay, Amazon marketplace, or a third-party seller — tampered devices with pre-generated seed phrases are a known and effective attack.
2. Initialise it yourself. It will show you a seed phrase. Write it on paper. Store it somewhere a burglar and a house fire would not both reach.

3. Bitcoin: use the device's built-in Bitcoin app. Solana: use the device together with Phantom (https://phantom.app), which supports hardware signing.
4. Send a **small test amount** first — $20 — and confirm it arrives before moving anything meaningful. Crypto addresses have no "are you sure?" and no recall.
5. This wallet is a vault, not part of the trading system. The scripts never see it and never need to.

8. Where Keys Live on the Server

Keys go in one file, readable only by the owning user, and are read from the environment. Never in the scripts, never in git.

Claude: create /home/ubuntu/portfolio/.env

```
# On the server. Note the permissions -- this matters.
install -m 700 -d /home/ubuntu/portfolio
cat > /home/ubuntu/portfolio/.env <<'EOF'
# Kraken -- trade permissions only, withdrawal DISABLED
KRAKEN_API_KEY=...
KRAKEN_API_SECRET=...

# Alpaca -- paper endpoint until Phase 3
ALPACA_API_KEY=...
ALPACA_API_SECRET=...
ALPACA_BASE_URL=https://paper-api.alpaca.markets

# Risk limits, enforced in code (see section on the execution layer)
MAX_ORDER_USD=500
SOL_CAP=0.25
LIVE_TRADING=0
EOF
chmod 600 /home/ubuntu/portfolio/.env
```

Rules for handling Kim's keys

1. `chmod 600`. Never world-readable.
2. Never committed to git. Add `.env` to `.gitignore` *before* the first commit, not after — git remembers.
3. Never echoed into a terminal, a log file, or an error message. Log the fact of a request, never its credentials.
4. Never pasted into a chat transcript, including this one.
5. Rotated if there is any doubt at all. Rotation is five minutes of Kim's time; a leak is permanent.
6. `LIVE_TRADING=0` is the master switch. The execution layer must refuse to place a real order unless it is explicitly `1`. Default closed.

Part III

Claude's Work

Everything from here is executable by Claude without Kim present.

9. SSH: Getting In

The existing instance is at `ubuntu@207.211.145.30`, with the alias already configured in `~/.ssh/config`:

/.ssh/config

```
Host oci
  HostName 207.211.145.30
  User ubuntu
  IdentityFile ~/.ssh/oci_new
```

Connect, and confirm what you are on

```
ssh oci # interactive
ssh oci 'uname -m; lsb_release -ds; python3 --version'
# expect: aarch64 / Ubuntu 24.04.4 LTS / Python 3.12.3
```

For a new instance (Option B), the key is generated locally and the public half is handed to Oracle at instance creation:

New host setup

```
ssh-keygen -t ed25519 -f ~/.ssh/kim_oci -C "kim-portfolio"
cat ~/.ssh/kim_oci.pub # paste into the OCI console

cat >> ~/.ssh/config <<'EOF'

Host kim-oci
  HostName <NEW_PUBLIC_IP>
  User ubuntu
  IdentityFile ~/.ssh/kim_oci
EOF

ssh kim-oci 'echo ok'
```

Two things that break new OCI instances

1. **Security lists.** OCI blocks inbound traffic by default at the *virtual network* level, independently of the host firewall. If SSH hangs rather than refusing, the ingress rule for port 22 is missing from the subnet's security list. Nothing on the host will fix this.
2. **Ubuntu's iptables.** Oracle's Ubuntu images ship with a restrictive `iptables` ruleset that surprises people who have already opened the security list. Check `sudo iptables -L INPUT -n` before concluding the network is at fault.

Outbound is open by default, which is all the scripts need.

10. Installing the Scripts

One command from the Mac. The installer is idempotent — re-running it is the normal way to deploy a change.

From the Mac

```
cd ~/git/andy/crypto/portfolio
bash install_oci.sh # or: OCI_HOST=kim-oci bash install_oci.sh
```

It performs, in order:

1. Verifies SSH reachability and fails loudly if not.
2. Creates `/home/ubuntu/portfolio/{reports,logs}`.
3. Copies `signals.py`, `weekly_report.py`, `run_weekly.sh`.
4. Builds a virtualenv and installs `numpy`, `pandas`, `matplotlib`, `yfinance`.
5. **Runs one report immediately**, so a broken install is discovered now rather than next Monday.
6. Installs the weekly cron entry.

ARM notes

The instance is `aarch64`. `numpy`, `pandas` and `matplotlib` all publish ARM wheels, so the install is a download rather than a compile — but it is slower than on x86 and wants `python3-dev` present in case anything does fall back to source. The installer handles this.
Use a virtualenv, not `pip install --user`: Ubuntu 24.04 enforces PEP 668 and will refuse to modify the system Python.
This ARM detail is also the concrete reason IBKR's Java gateway is awkward here (Section 5) — its installer bundles an x86 runtime.

11. The Cron Job

The house style on this box — visible in the ten existing entries — is absolute paths and appended logs. We match it.

`crontab -e`

```
# Recycling portfolio weekly report
# Sunday 21:15 UTC = Monday 07:15 AEST -- before the working week,
# after the US Friday close has settled into the data.
15 21 * * 0 /home/ubuntu/portfolio/run_weekly.sh >> /home/ubuntu/portfolio/logs/cron.log 2>&1
```

The installer adds this for you, stripping any previous copy of the same line first so repeated runs cannot produce duplicates.

Cron facts that cause silent failures

1. **Cron's `PATH` is nearly empty** and it does not read your shell profile. Every path in a cron job must be absolute — including the Python interpreter. This is why `run_weekly.sh` invokes `./venv/bin/python` rather than `python3`.
2. **The server runs UTC.** Confirm with `date -u` and convert deliberately; do not assume local time.
3. **Without redirection, output is emailed** — and on a box with no mail transport, silently discarded. Always `>> logfile 2>&1`.
4. **A cron job that fails is invisible.** Check `grep CRON /var/log/syslog` for whether it fired at all, and the job's own log for what happened when it did.
5. **Test without waiting a week:** run `/home/ubuntu/portfolio/run_weekly.sh` by hand, then verify with `env -i /bin/bash -c '/home/ubuntu/portfolio/run_weekly.sh'`, which reproduces cron's bare environment. Most cron bugs appear only under that second form.

12. The Heartbeat: Not Being Reclaimed

Oracle gives away Always Free instances, and reserves the right to take back the ones nobody is using. A box that runs a script for four seconds a week looks, to a monitoring system, exactly like a box nobody is using.

Oracle's idle-reclamation rule, and the loophole in it

An Always Free compute instance becomes a candidate for reclamation when, across a **7-day window**, **all** of the following hold:

Metric (95th percentile)	Threshold
CPU utilisation	$< 20\%$
Network utilisation	$< 20\%$
Memory utilisation (Ampere A1 only)	$< 20\%$

The important word is **all**. The test is a conjunction, so **defeating any single criterion is sufficient** — there is no need to hold three metrics up. CPU is by far the cheapest to move, so that is the one we move.

12.1 How the keepalive works

`keepalive.sh` runs every 30 minutes and burns one core for five minutes. That puts roughly a sixth of all samples near 100% on one core — comfortably enough to hold the 95th percentile above 20%, with a wide margin for a missed run.

It burns that core at `nice -n 19` and `ionice -c3`, the lowest possible scheduling priority for both CPU and disk. In practice the loop is descheduled the instant anything real wants the processor: the four AI agents and the Gateway app never wait on it. We are consuming *idle* capacity — which is exactly the capacity Oracle is measuring.

The cron pair

```
# Recycling portfolio: weekly report + keep instance from being reclaimed
15 21 * * 0 /home/ubuntu/portfolio/run_weekly.sh >> /home/ubuntu/portfolio/logs/cron.log 2>&1
*/30 * * * * /home/ubuntu/portfolio/keepalive.sh >> /home/ubuntu/portfolio/logs/keepalive.log 2>&1
```

Robin's box probably does not need this. Install it anyway.

The existing instance runs four AI agents, the Gateway app and a `*/15` cron job, so its CPU is unlikely to look idle. The keepalive is insurance, and it is nearly free.

It matters more than it looks, because of what happens on the day it is needed: the agents go quiet during a lull, Oracle's 7-day window closes, the instance is stopped, and **the weekly reports simply stop arriving.** Nothing errors. Nothing alerts. The failure mode is *silence* — which is precisely the failure mode a report you glance at once a week is worst at revealing.

12.2 The other heartbeat: knowing it is alive

Two different things get called a heartbeat, and we want both. The keepalive above beats *outward*, convincing Oracle we exist. We also want one beating *inward*, convincing us the job still runs.

So `keepalive.sh` writes `heartbeat.txt` every 30 minutes *before* it burns anything, so the timestamp lands even if the burn is killed. `healthcheck.sh` reads that file, plus the report directory and the crontab, and answers the only question that matters:

Run this whenever you wonder

```
$ssh oci '/home/ubuntu/portfolio/healthcheck.sh'

Portfolio health 2026-08-27T00:15:02Z

  keepalive ok - 12 min ago
  weekly report ok - 1 day(s) old
  report count 1 archived
  signals no data errors
  signals mode: paper
  cron: report installed
  cron: keepalive installed
  disk 69G free of 145G (53% used)

HEALTHY
```

It exits non-zero when anything needs attention, so it composes with anything that watches exit codes. The checks that fire in practice: a stale keepalive (instance stopped, or cron broken), an overdue report (more than 8 days old), and data-feed errors recorded by the signal engine.

Free-tier hygiene, restated because it is the only thing that costs money

Compute on the Always Free tier is free and stays free. **Storage is capped at 200 GB across the entire tenancy — including volumes attached to nothing at all.** On 3 June 2026 four abandoned 47 GB rescue boot volumes pushed Robin's tenancy to 338 GB and produced roughly AUD $8/month of charges on what should have been a $0 bill. `healthcheck.sh` reports disk usage for this reason. If a bill ever appears, look for orphaned volumes first — the commands are in Section 21.

13. The Signal Engine

`signals.py` computes each asset's momentum ratio with arithmetic *identical* to `momentum_single.py`, so live signals and backtested signals cannot silently diverge. It then applies the recycling rule with the Solana cap.

Verified output, run 2026-08-27

```
$./venv/bin/python signals.py

Signals as of 2026-08-26T21:32:07+00:00 (3/4 active)

Asset Symbol Close Ratio Signal Weight Trade as
------------------------------------------------------------------------
COPPER HG=F 6.70 0.817 cash 0.0% CPER @ IBKR
NATGAS NG=F 2.90 1.102 HOLD 37.5% UNG @ IBKR
BTC BTC-USD 78,659.15 1.184 HOLD 37.5% XBT/USD @ KRAKEN
SOL SOL-USD 97.59 1.302 HOLD 25.0% SOL/USD @ KRAKEN (capped)
------------------------------------------------------------------------
CASH 0.0%
```

Note that the cap is binding in this very sample: three assets are active, so equal weighting would give Solana 33.3%, and the cap holds it to 25% with the excess going to copper and gas. The mechanism is not hypothetical.

13.1 Two deliberate design decisions

1. **Fail-safe on missing data.** If a price feed fails, the asset's signal is recorded as *off* and the money stays in cash. A data outage can therefore cost us an opportunity; it can never cause an accidental position. Errors are listed in the report so a silent outage cannot masquerade as a genuine "all clear."
2. **Signal instrument ≠ traded instrument.** Momentum is computed on `HG=F` and `NG=F` — exactly what the study measured — while orders would be placed in `CPER`/`UNG`. Conflating the two would quietly invalidate the research, so the mapping is explicit in the `ASSETS` table.

14. The Execution Layer (Not Yet Written)

This is the only substantial piece of code still outstanding, and it should not be written until Phase 2 has run clean. The specification:

1. Read target weights from `signals.py`.
2. Read actual holdings from each venue.
3. Compute the difference, and **only trade if a position is off target by more than a threshold** (5% of portfolio value is a sensible starting point). Without this, tiny daily drift generates constant fee-burning churn.

4. Place market orders, sized to the smaller of the required trade and `MAX_ORDER_USD`.
5. Refuse to do anything at all unless `LIVE_TRADING=1`. Log the orders it *would* have placed otherwise — that log is how we validate it.
6. Write `accounts.json` after every run, which is what the weekly report reads to reconcile paper against real.

Sketch: reading balances from Kraken via ccxt

```
import os, ccxt

kraken = ccxt.kraken({
    "apiKey": os.environ["KRAKEN_API_KEY"],
    "secret": os.environ["KRAKEN_API_SECRET"],
    "enableRateLimit": True,
})

balance = kraken.fetch_balance() # needs "Query funds" only
print({k: v for k, v in balance["total"].items() if v})

# Placing an order needs "Create & modify orders". Guarded:
if os.environ.get("LIVE_TRADING") != "1":
    raise SystemExit("LIVE_TRADING is not 1 -- refusing to place orders")
```

Build it in this order, and not another

1. Read-only first. Fetch balances and prices for a week. Confirm the numbers match what Kim sees when he logs in.
2. Then dry-run orders — computed, logged, never sent — for a week. Read every line.
3. Then live orders capped at $50, for a week.
4. Then raise the cap.

Each stage catches a category of bug the next stage would make expensive. Skipping stage 1 is how people discover their position sizing was wrong by a factor of a hundred.

Part IV

The Weekly Report

15. What It Produces

Every Monday morning the server writes a two-page PDF and a JSON summary. The PDF is rendered with `matplotlib`, deliberately *not* LaTeX — the server needs no TeX installation, and there is no compile step that can fail at 7am.

Page 1: headline tiles (NAV, weekly change, return since inception, maximum drawdown); the signal table with each asset's ratio, target weight and traded instrument; a bar showing where the money should be right now; and a notes block flagging paper-versus-live mode, data errors, and the Solana cap.

Page 2: the NAV curve since inception, the drawdown curve, and a timeline strip showing which assets were held when.

Verified output, run 2026-08-27

```
$./venv/bin/python weekly_report.py --outdir ~/portfolio/reports
Wrote /home/ubuntu/portfolio/reports/portfolio-2026-W35.pdf
  Paper NAV $11,862.83 week +10.81% total +18.63% maxDD -2.9%
  Mode: paper Active: 3/4
```

16. The Design Decision That Matters

The report is stateless

Every run rebuilds the *entire* equity curve from scratch, from first principles, back to inception. Nothing is carried forward between runs.

This costs a few seconds of ARM CPU per week, and buys a property worth far more than that: **a missed week, a crashed run, a truncated file, or a server rebuild cannot cause the tracked NAV to drift.** An incrementally-updated ledger accumulates errors silently and is only discovered when the number is already wrong. A recomputed one is self-healing — correct this week regardless of what happened last week.

The same reasoning is why `run_weekly.sh` keeps the last 52 reports rather than overwriting: the history is evidence, and evidence should be append-only.

17. How Claude Fetches It

Three ways, in increasing order of convenience:

Getting at the report

```
# 1. The one-liner Claude will normally use: numbers, no file transfer.
ssh oci 'cat /home/ubuntu/portfolio/reports/latest.json'

# 2. Copy the PDF back and open it.
bash ~/git/andy/crypto/portfolio/fetch_report.sh # latest
bash ~/git/andy/crypto/portfolio/fetch_report.sh 2026-W35 # a given week

# 3. Look around.
ssh oci 'ls -lt /home/ubuntu/portfolio/reports/ | head'
ssh oci 'tail -40 /home/ubuntu/portfolio/logs/cron.log'
```

`latest.json` exists precisely so Claude does not have to parse a PDF to answer "how is it going?". It carries the week, the mode (paper or live), every ratio, every weight, and the summary statistics. `fetch_report.sh` prints a one-line summary and opens the PDF in Skim.

latest.json, abridged

```
{
  "week": "2026-W35",
  "mode": "paper",
  "stats": { "nav": 11862.83, "week_return": 0.1081,
             "total_return": 0.1863, "max_dd": -0.029, "sharpe": ... },
  "signals": { "COPPER": false, "NATGAS": true, "BTC": true, "SOL": true },
  "ratios": { "COPPER": 0.817, "NATGAS": 1.102, "BTC": 1.184, "SOL": 1.302 },
  "weights": { "COPPER": 0.0, "NATGAS": 0.375, "BTC": 0.375, "SOL": 0.25 },
  "cash": 0.0, "errors": []
}
```

Part V

Going Live

18. Choosing the Solana Cap

This was an open question when the runbook was drafted, and it is now closed. `cap_sweep.py` regenerates momentum for all four assets from source prices — the saved `timing_*.csv` files could not be used, see the warning below — builds the recycling portfolio for a range of caps, and reports

the trade-off.

Period 19 April 2021 to 26 August 2026 (1,956 days, 5.35 years), $10,000 start, signals lagged one day, no fees.

SOL cap	Final	CAGR	Sharpe	Sortino	Max DD
0% (no SOL)	$95,575	52.5%	1.26	1.43	−35.3%
10%	$119,632	59.0%	1.37	1.71	−35.1%
15%	$132,434	62.0%	1.41	1.79	−35.2%
17.5%	$138,973	63.5%	1.42	1.82	−35.3%
20%	$145,579	64.9%	1.43	1.86	−36.3%
25%	**$158,908**	67.7%	1.44	1.91	−38.7%
33.3%	$178,679	71.4%	1.44	1.94	−42.8%
50%	$169,259	69.6%	1.35	1.81	−50.6%
no cap	$114,144	57.6%	1.08	1.37	−75.4%

The result nobody expected: the cap is not a trade-off

The uncapped rule is the worst row in the table. Not the riskiest-but-highest-returning — the worst, on both axes at once. It makes less money than a 10% cap and takes double the drawdown. *Every* cap between 10% and 50% dominates it.

The mechanism is volatility drag. Solana signals alone on 24.1% **of its active days**, and on those days the uncapped rule puts the entire portfolio into an asset that has historically fallen 96%. Occasionally concentrating everything into that destroys the compounding, however good the asset is on average. This is the −50%-then-+50% arithmetic from `concepts.pdf`, operating at portfolio scale.

So the cap does not buy safety at the cost of return. **It buys both.**

18.1 Why 25%, given the plateau

Sharpe is essentially flat from 20% to 33% (1.43 to 1.45) — those differences are noise, not signal. Return keeps climbing to 33%; drawdown climbs with it, and faster beyond 25%. So the choice inside the plateau is a judgement, and it should not be made by picking the maximum of anything.

A split-half check is the reason for caution:

	First half (to Dec 2023)	Second half (to Aug 2026)
Best Sharpe	33%	15%
Best Calmar	33%	27.5%
Best return	33%	50%
Uncapped	disastrous (−75% DD, Sharpe 0.83)	barely differs from capped

The two halves disagree about the optimum and agree about everything that matters: caps in the 15–33% band beat both extremes, and uncapped is only catastrophic in the half that contained a crash. A number tuned to the full-period maximum would be fitting the 2022 drawdown.

Decision: 25%

1. **It is not a fitted parameter.** 25% is the natural equal share of four assets. The cap therefore only ever binds *downward* — it can never force us to hold more Solana than plain diversification would. That is the strongest available defence against overfitting: the

number comes from the portfolio's structure, not from the data.
2. It sits inside the flat region of the Sharpe curve, so it gives up nothing measurable in risk-adjusted terms.
3. It adds 66% to the final value over the three-asset portfolio ($95,575 → $158,908) while moving maximum drawdown only from −35.3% to −38.7%.
4. Both halves of the sample support it.

If Kim wants a gentler ride, 20% is the defensible alternative: the best Calmar ratio in the table, drawdown a little over a point shallower, at the cost of about 8% of the final value. That is a preference, not a correction — both are right.

Do not raise it above 33%**.** Past that, drawdown grows and return stops improving. `SOL_CAP` in `.env` exists so this is one line to change, and `signals.py` enforces it on every run.

The data problem this uncovered

The saved `timing_*.csv` files in `~/git/andy/crypto/` **are stale and inconsistent with the published study**, and any analysis built on them would have been confidently wrong:

File	Contains	Study says	
`timing_BTCUSD.csv`	+462.8%	+462.7%	matches
`timing_SOLUSD.csv`	+201.8%	+7,691%	365-day run
`timing_HG=F.csv`	+15.0%	+97.0%	wrong run
`timing_NG=F.csv`	+88.4%	+309.9%	wrong run

The Solana file holds the **365-day** lookback — the one variant that produced *negative* alpha — not the 180-day run the study recommends. Copper and gas match neither.

`cap_sweep.py` therefore ignores those files entirely and refetches prices into its own `cache/` directory. Leftover result files from a parameter sweep are not a data source. They should be regenerated or deleted before they mislead someone.

19. Four Phases, With Gates

Each phase has an exit criterion. The point of a gate is that it can fail — if a criterion is not met, the answer is to stop, not to proceed while noting a concern.

Phase	What runs	Kim does	Gate to pass
1 1 week	Reports on Robin's box, paper only, no keys	Step 1 (Oracle); nothing else	A report lands unattended on Monday; signals match a hand-check
2 4–12 weeks	Execution layer dry-run; ETF tracking measured vs futures	Steps 2–3 (accounts, paper keys)	Dry-run orders sane every week; backtest confirms the cap; ETF decay measured
3 4 weeks	Live crypto leg only, $50 order cap	Step 4 (fund small), live keys	Broker NAV tracks paper NAV within a few percent; no failed orders
4 ongoing	Full size; copper/-gas added if Phase 2 justified it	Increase funding; hardware wallet	Reviewed quarterly against the mind-changers below

The gate on Phase 2 is the one that matters

One of the two gaps here is now closed. The four-asset backtest with the Solana cap has been run — Section 18 — and it supports the design: capped at 25%, $10,000 becomes $158,908 over 5.35 years with a −38.7% worst drawdown, against $95,575 for the three-asset portfolio. Uncapped is worse on *both* counts.

What Phase 2 still has to establish is the part no backtest can tell us: **how much of that survives in instruments we can actually buy.** The signals are measured on `HG=F` and `NG=F`; the orders would be in `CPER` and `UNG`, which carry roll decay the backtest never saw. Until that difference is measured on live prices, the copper and gas legs stay on paper.

20. What Would Stop Us

Stated in advance, so they cannot be rationalised away later:

- ETF tracking error eats the copper or gas alpha ⇒ drop that leg rather than trade it hoping.
- Signal overlap between the four assets turns out to be high ⇒ the recycling rationale is void; rebuild the allocation logic.
- UNG's roll decay eats the natural gas alpha ⇒ drop the gas leg or find a different instrument. Do not trade it hoping.
- Live results diverge materially from paper over a quarter ⇒ stop and find out why before adding money.
- Results prove sensitive to the lookback window ⇒ treat the whole thing as overfitted. A real effect should not require the knob set to exactly 180.
- Kim cannot comfortably tolerate a 30% drawdown in practice rather than in principle ⇒ reduce size until he can. This is a legitimate reason and needs no other justification.

21. Command Reference

Everything, in one place

```
# --- Deploy (from the Mac) -------------------------------------------
cd ~/git/andy/crypto/portfolio
bash install_oci.sh # install + venv + cron + smoke test
OCI_HOST=kim-oci bash install_oci.sh # ... to a different host

# --- Check on it -----------------------------------------------------
ssh oci 'cat /home/ubuntu/portfolio/reports/latest.json'
ssh oci 'cd /home/ubuntu/portfolio && ./venv/bin/python signals.py'
ssh oci 'tail -40 /home/ubuntu/portfolio/logs/cron.log'
ssh oci 'crontab -l | grep -A2 Recycling'
ssh oci '/home/ubuntu/portfolio/healthcheck.sh' # alive and on schedule?
ssh oci 'cat /home/ubuntu/portfolio/heartbeat.txt' # last keepalive
ssh oci 'grep CRON /var/log/syslog | tail -5'

# --- Reports ---------------------------------------------------------
bash fetch_report.sh # newest, opens in Skim
bash fetch_report.sh 2026-W35 # a specific week
ssh oci 'ls -lt /home/ubuntu/portfolio/reports | head'

# --- Run out of band -------------------------------------------------
ssh oci '/home/ubuntu/portfolio/run_weekly.sh'
ssh oci "env -i /bin/bash -c '/home/ubuntu/portfolio/run_weekly.sh'" # as cron sees it

# --- Free-tier hygiene (do this after any OCI console work) ----------
# Orphaned volumes are the only way this costs money. 200 GB total cap.
oci bv boot-volume list --compartment-id <tenancy-ocid> \
   --availability-domain "drxR:AP-MELBOURNE-1-AD-1" --all
oci bv volume list --compartment-id <tenancy-ocid> --all
```

File	Purpose
`signals.py`	Momentum ratios for the four assets; recycling weights with the Solana cap. Arithmetic identical to `momentum_single.py`.
`weekly_report.py`	Rebuilds the paper equity curve from scratch; renders the two-page PDF, `latest.pdf` and `latest.json`.
`run_weekly.sh`	Cron entry point. Absolute paths, logs, prunes to 52 reports.
`install_oci.sh`	Idempotent deploy from the Mac: copy, venv, smoke test, cron.
`fetch_report.sh`	Pull the latest PDF back to the Mac and open it.
`keepalive.sh`	Burns one core for 5 min every 30 min so Oracle does not reclaim the instance; writes `heartbeat.txt`.
`healthcheck.sh`	One command answering "is it alive and on schedule?". Non-zero exit if not.
`cap_sweep.py`	Regenerates momentum from source data and sweeps the Solana cap. Produced the numbers in Section 18.
`make_figure.py`	Self-contained: fetches prices and draws the three-way portfolio comparison. Listed in full in `algorithm.pdf`.
`.env` (server only)	API keys and risk limits. `chmod 600`, never in git.
`execute.py`	Not yet written. Order placement. Phase 2.

Nothing in this document is financial advice.

It is a deployment runbook for a strategy that has been backtested and not yet validated live. The strategy's own caveats are in `concepts.pdf` and `kim.pdf`, and they are substantial. Read them before Step 4.

Part VI

Appendix: The Complete Code

Everything Claude needs to implement the copper–gas–bitcoin–sol strategy is listed here in full. Nothing is elided.

These listings cannot go stale

The listings below are pulled from the working files at compile time via `\lstinputlisting`. They are not copies. If a script changes and this document is rebuilt, the appendix changes with it — so the PDF and the code can never silently disagree.

Source of truth: `~/git/andy/crypto/portfolio/`. Recompile with `tectonic oci.tex` after any edit.

File	Lines	Role
`signals.py`	229	Momentum ratios; recycling weights with the SOL cap
`weekly_report.py`	413	Paper equity curve; the two-page PDF report
`cap_sweep.py`	213	Cap selection backtest (Section 18)
`run_weekly.sh`	14	Cron entry point
`keepalive.sh`	47	Oracle idle-reclamation defence + heartbeat
`healthcheck.sh`	83	Liveness and schedule check
`install_oci.sh`	72	One-command deploy from the Mac
`fetch_report.sh`	25	Pull the latest report back
Total	**1,096**	

What is deliberately NOT here

`execute.py` — the code that places real orders — is not in this appendix because it has not been written, and it should not be written until Phase 2 has run clean. Its specification is in Section 14, and the build order given there (read-only → dry-run → $50 cap → full size) matters more than the code itself.

Also absent: `.env`, which holds the API keys. It is created on the server by hand, `chmod 600`, and never enters git or any document. See Section 8.

22. signals.py

The heart of the system. The momentum arithmetic here is byte-for-byte equivalent to `momentum_single.py`, which is what makes live signals comparable with the backtest. Note two deliberate choices: a missing price feed produces *cash*, never a position; and the signal instrument is kept distinct from the traded instrument.

```
"""
Signal engine for the copper-gas-bitcoin-sol recycling portfolio.

Computes today's momentum signal for each of the four assets using
Robin's bookmaker-pool model (identical arithmetic to
momentum_single.py, so live signals match the backtest), then turns
```

```
the signals into target portfolio weights under the capital-recycling
rule with a hard cap on SOL.

Signals are computed on the *backtested* instrument (e.g. HG=F front-month
copper futures). Orders are placed in the *tradeable* instrument
(e.g. the CPER ETF). See TRADEABLE below.

Usage:
    python3 signals.py # print today's signals + weights
    python3 signals.py --json # machine-readable
    python3 signals.py --history 30 # last 30 days of signals
"""

import argparse
import json
import sys
from datetime import datetime, timedelta, timezone

import numpy as np
import pandas as pd
import yfinance as yf

# -- Portfolio definition ------------------------------------------
# signal_symbol: what we compute momentum on (matches the backtest)
# lookback: window in days (per-asset optimum from the study)
# cap: maximum share of the portfolio, None = uncapped
# tradeable: what we actually buy (venue-dependent, see oci.pdf)

ASSETS = {
    "COPPER": {"signal_symbol": "HG=F", "lookback": 180, "cap": None,
               "tradeable": "CPER", "venue": "IBKR"},
    "NATGAS": {"signal_symbol": "NG=F", "lookback": 90, "cap": None,
               "tradeable": "UNG", "venue": "IBKR"},
    "BTC": {"signal_symbol": "BTC-USD", "lookback": 180, "cap": None,
               "tradeable": "XBT/USD", "venue": "KRAKEN"},
    "SOL": {"signal_symbol": "SOL-USD", "lookback": 180, "cap": 0.25,
               "tradeable": "SOL/USD", "venue": "KRAKEN"},
}

ORDER = ["COPPER", "NATGAS", "BTC", "SOL"]


# -- Core algorithm (unchanged from momentum_single.py) ----------------

def momentum_ratio(history, lookback_days):
    """
    Bookmaker-pool momentum ratio for the most recent day.

    Each day's net flow is (volume / mean volume) * return. Flows are
    compounded into a pool seeded at the price at the start of the
    window. The result is the 'momentum price'; we return its ratio to
    the actual close.

    Returns (ratio, close) or (None, close) if there is not enough data.
    """
    closes = history["Close"].values.astype(float)
    volumes = history["Volume"].values.astype(float)

    if len(closes) < lookback_days + 1:
        return None, (closes[-1] if len(closes) else None)

    window_close = closes[-(lookback_days + 1):]
    window_vol = volumes[-(lookback_days + 1):]

    returns = np.diff(window_close) / window_close[:-1]
    vols = window_vol[1:]

    mean_vol = vols.mean()
    if mean_vol == 0 or not np.isfinite(mean_vol):
        return None, closes[-1]
    norm_vols = vols / mean_vol

    net_flows = norm_vols * returns

    pool = float(window_close[0])
    mp = pool
    for flow in net_flows:
        pool += flow * mp
        mp = pool

    if not np.isfinite(mp):
        return None, closes[-1]

    return mp / closes[-1], closes[-1]


# -- Allocation ----------------------------------------------------

def target_weights(signals):
    """
    Capital-recycling weights with per-asset caps.

    Equal weight across every asset whose signal is on. Any weight above
    an asset's cap is redistributed to the other active, uncapped assets;
    if there are none, it stays in cash.

    signals: {asset: bool}
    Returns {asset: weight}; weights sum to <= 1, remainder is cash.
    """
    active = [a for a in ORDER if signals.get(a)]
    if not active:
```

```
        return {a: 0.0 for a in ORDER}

    w = {a: (1.0 / len(active) if a in active else 0.0) for a in ORDER}

    # Apply caps, spilling the excess onto uncapped active assets.
    for _ in range(len(active)): # iterate to a fixed point
        excess = 0.0
        for a in active:
            cap = ASSETS[a]["cap"]
            if cap is not None and w[a] > cap + 1e-12:
                excess += w[a] - cap
                w[a] = cap
        if excess <= 1e-12:
            break
        receivers = [a for a in active if ASSETS[a]["cap"] is None]
        if not receivers:
            break # excess stays in cash
        for a in receivers:
            w[a] += excess / len(receivers)

    return w


def fetch_signals(asof=None):
    """Fetch data and compute signals for all four assets."""
    end = (asof or datetime.now(timezone.utc)) + timedelta(days=1)
    out = {}
    for name in ORDER:
        cfg = ASSETS[name]
        lb = cfg["lookback"]
        start = end - timedelta(days=lb + 200) # slack for weekends/holidays
        try:
            hist = yf.Ticker(cfg["signal_symbol"]).history(
                start=start.date(), end=end.date(), interval="1d",
                auto_adjust=False,
            )
            hist = hist.dropna(subset=["Close", "Volume"])
            ratio, close = momentum_ratio(hist, lb)
            out[name] = {
                "symbol": cfg["signal_symbol"],
                "lookback": lb,
                "close": None if close is None else float(close),
                "ratio": None if ratio is None else float(ratio),
                "signal": bool(ratio is not None and ratio > 1.0),
                "asof": (str(hist.index[-1].date()) if len(hist) else None),
                "cap": cfg["cap"],
                "tradeable": cfg["tradeable"],
                "venue": cfg["venue"],
                "error": None,
            }
        except Exception as exc: # noqa: BLE001
            out[name] = {
                "symbol": cfg["signal_symbol"], "lookback": lb,
                "close": None, "ratio": None, "signal": False, "asof": None,
                "cap": cfg["cap"], "tradeable": cfg["tradeable"],
                "venue": cfg["venue"], "error": str(exc),
            }
    return out


def build_state(asof=None):
    """Full state dict: signals, weights, metadata."""
    sig = fetch_signals(asof)
    weights = target_weights({k: v["signal"] for k, v in sig.items()})
    cash = max(0.0, 1.0 - sum(weights.values()))
    return {
        "generated": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "assets": sig,
        "weights": weights,
        "cash": cash,
        "n_active": sum(1 for v in sig.values() if v["signal"]),
        "errors": [k for k, v in sig.items() if v["error"]],
    }


def render_text(state):
    lines = []
    lines.append(f"Signals as of {state['generated']} "
                 f"({state['n_active']}/4 active)")
    lines.append("")
    lines.append(f"{'Asset':8s} {'Symbol':9s} {'Close':>12s} {'Ratio':>8s} "
                 f"{'Signal':>7s} {'Weight':>8s} Trade as")
    lines.append("-" * 76)
    for a in ORDER:
        v = state["assets"][a]
        close = "n/a" if v["close"] is None else f"{v['close']:,.2f}"
        ratio = "n/a" if v["ratio"] is None else f"{v['ratio']:.3f}"
        sig = "HOLD" if v["signal"] else "cash"
        cap = " (capped)" if (v["cap"] is not None
                              and state["weights"][a] >= v["cap"] - 1e-9
                              and v["signal"]) else ""
        lines.append(f"{a:8s} {v['symbol']:9s} {close:>12s} {ratio:>8s} "
                     f"{sig:>7s} {state['weights'][a]*100:7.1f}% "
                     f"{v['tradeable']} @ {v['venue']}{cap}")
    lines.append("-" * 76)
    lines.append(f"{'CASH':8s} {'':9s} {'':>12s} {'':>8s} {'':>7s} "
                 f"{state['cash']*100:7.1f}%")
    if state["errors"]:
        lines.append("")
        lines.append(f"WARNING: no data for {', '.join(state['errors'])} "
                     f"- treated as 'cash' (fail-safe).")
```

```
    return "\n".join(lines)


def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--json", action="store_true", help="JSON output")
    p.add_argument("--out", default=None, help="write JSON state to this path")
    args = p.parse_args()

    state = build_state()

    if args.out:
        with open(args.out, "w") as fh:
            json.dump(state, fh, indent=2)

    print(json.dumps(state, indent=2) if args.json else render_text(state))
    return 1 if len(state["errors"]) == len(ORDER) else 0


if __name__ == "__main__":
    sys.exit(main())
```

23. weekly_report.py

Rebuilds the entire paper equity curve from scratch on every run, then renders the report with `matplotlib` — no LaTeX on the server, so there is no compile step that can fail at 7am.

```
"""
Weekly PDF report for the copper-gas-bitcoin-sol recycling portfolio.

Rebuilds the whole paper-trading equity curve from scratch on every run,
using the same bookmaker-pool arithmetic as the backtest. That makes the
report stateless and self-healing: a missed week, a crashed run, or a
corrupted file cannot cause the tracked NAV to drift, because nothing is
carried forward.

If a broker snapshot exists (accounts.json, written by the execution
layer once live trading starts), the real NAV is reported alongside the
paper NAV so the two can be compared.

Outputs, all under --outdir:
    portfolio-YYYY-Www.pdf this week's report
    latest.pdf copy of the most recent report
    latest.json machine-readable summary (for Claude)
    nav_history.csv daily paper NAV series

Usage:
    python3 weekly_report.py # default paths
    python3 weekly_report.py --capital 10000
    python3 weekly_report.py --outdir ~/portfolio/reports
"""

import argparse
import json
import os
import shutil
from datetime import datetime, timedelta, timezone

import numpy as np
import pandas as pd
import yfinance as yf

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages

from signals import ASSETS, ORDER, target_weights, build_state

INCEPTION = "2026-08-01" # when paper tracking starts
ACCENT = "#1e4678"
GREEN = "#287828"
RED = "#b42828"
GREY = "#888888"
PALETTE = {"COPPER": "#b87333", "NATGAS": "#4c9be8",
           "BTC": "#f2a900", "SOL": "#9945ff"}


# -- Paper equity curve ------------------------------------------------

def rolling_ratios(hist, lookback):
    """Momentum ratio for every day with a full lookback window."""
    closes = hist["Close"].values.astype(float)
    volumes = hist["Volume"].values.astype(float)
    out = np.full(len(closes), np.nan)

    for i in range(lookback, len(closes)):
        wc = closes[i - lookback:i + 1]
        wv = volumes[i - lookback:i + 1]
        rets = np.diff(wc) / wc[:-1]
        vols = wv[1:]
        mv = vols.mean()
        if mv == 0 or not np.isfinite(mv):
            continue
        flows = (vols / mv) * rets
        pool = float(wc[0])
```

```
        mp = pool
        for f in flows:
            pool += f * mp
            mp = pool
        if np.isfinite(mp) and closes[i] != 0:
            out[i] = mp / closes[i]

    return pd.Series(out, index=hist.index)


def build_paper_curve(capital, inception=INCEPTION):
    """
    Daily paper NAV since inception.

    Signals use the previous day's ratio (you cannot trade on a close you
    have not seen yet) - the same one-day lag as the backtest.
    """
    start = pd.Timestamp(inception)
    fetch_from = (start - timedelta(days=400)).date()
    end = (datetime.now(timezone.utc) + timedelta(days=1)).date()

    ratios, rets, closes = {}, {}, {}
    for name in ORDER:
        cfg = ASSETS[name]
        h = yf.Ticker(cfg["signal_symbol"]).history(
            start=fetch_from, end=end, interval="1d", auto_adjust=False)
        h = h.dropna(subset=["Close", "Volume"])
        if h.empty:
            continue
        h.index = pd.to_datetime(h.index, utc=True).tz_localize(None).normalize()
        h = h[~h.index.duplicated(keep="last")]
        ratios[name] = rolling_ratios(h, cfg["lookback"])
        rets[name] = h["Close"].pct_change()
        closes[name] = h["Close"]

    if not ratios:
        return None, None, None

    R = pd.DataFrame(ratios).sort_index()
    P = pd.DataFrame(rets).reindex(R.index)

    # Trade on yesterday's signal.
    S = (R > 1.0) & R.notna()
    S = S.shift(1)
    S = S.where(S.notna(), False).astype(bool)

    idx = R.index[R.index >= start]
    if len(idx) == 0:
        return None, None, None
    S, P = S.reindex(idx).fillna(False), P.reindex(idx).fillna(0.0)

    W = pd.DataFrame(0.0, index=idx, columns=ORDER)
    for d in idx:
        w = target_weights({a: bool(S.get(a, pd.Series(dtype=bool)).get(d, False))
                            for a in ORDER})
        for a in ORDER:
            W.at[d, a] = w[a]

    port = (W * P.reindex(columns=ORDER).fillna(0.0)).sum(axis=1)
    nav = capital * (1 + port).cumprod()
    return nav, W, closes


def stats(nav):
    if nav is None or len(nav) < 2:
        return {}
    r = nav.pct_change().dropna()
    eq = nav / nav.iloc[0]
    dd = (nav / nav.cummax() - 1)
    week_ago = nav.index[-1] - timedelta(days=7)
    prior = nav[nav.index <= week_ago]
    wk = (nav.iloc[-1] / prior.iloc[-1] - 1) if len(prior) else np.nan
    days = max(1, (nav.index[-1] - nav.index[0]).days)
    return {
        "nav": float(nav.iloc[-1]),
        "start_nav": float(nav.iloc[0]),
        "total_return": float(eq.iloc[-1] - 1),
        "week_return": float(wk) if wk == wk else None,
        "annualised": float(eq.iloc[-1] ** (365.0 / days) - 1),
        "sharpe": (float(r.mean() / r.std() * np.sqrt(252))
                   if r.std() > 0 else None),
        "max_dd": float(dd.min()),
        "current_dd": float(dd.iloc[-1]),
        "days": int(days),
        "deployed_pct": None,
    }


# -- PDF rendering -----------------------------------------------------

def _txt(ax, x, y, s, size=10, weight="normal", color="black", family=None,
         ha="left", va="top"):
    ax.text(x, y, s, size=size, weight=weight, color=color, ha=ha, va=va,
            family=family, transform=ax.transAxes)


def render_pdf(path, state, st, nav, W, broker):
    week = datetime.now(timezone.utc).strftime("%G-W%V")
    with PdfPages(path) as pdf:

        # -- Page 1: summary ---------------------------------------
```

```
fig = plt.figure(figsize=(8.27, 11.69))
fig.subplots_adjust(left=0.08, right=0.94, top=0.95, bottom=0.06)

head = fig.add_axes([0.08, 0.83, 0.86, 0.12]); head.axis("off")
_txt(head, 0, 1.0, "Recycling Portfolio", 22, "bold", ACCENT)
_txt(head, 0, 0.55, "Copper | Natural Gas | Bitcoin | Solana",
     12, color=GREY)
_txt(head, 0, 0.22,
     f"Week {week} | generated "
     f"{datetime.now(timezone.utc):%Y-%m-%d %H:%M} UTC", 9, color=GREY)

# Headline tiles
tiles = fig.add_axes([0.08, 0.70, 0.86, 0.11]); tiles.axis("off")
if st:
    wk = st.get("week_return")
    items = [
        ("Paper NAV", f"${st['nav']:,.0f}", ACCENT),
        ("This week", "n/a" if wk is None else f"{wk*100:+.2f}%",
         GREEN if (wk or 0) >= 0 else RED),
        ("Since inception", f"{st['total_return']*100:+.2f}%",
         GREEN if st["total_return"] >= 0 else RED),
        ("Max drawdown", f"{st['max_dd']*100:.1f}%", RED),
    ]
    for i, (lab, val, col) in enumerate(items):
        x = i * 0.25
        tiles.add_patch(plt.Rectangle((x + 0.005, 0.05), 0.24, 0.9,
                                      transform=tiles.transAxes,
                                      facecolor="#f2f5fa",
                                      edgecolor="#d5dee9"))
        _txt(tiles, x + 0.02, 0.80, lab, 8, color=GREY)
        _txt(tiles, x + 0.02, 0.52, val, 15, "bold", col)
else:
    _txt(tiles, 0, 0.6, "No price history available this run.", 11, color=RED)

# Signal table
tab = fig.add_axes([0.08, 0.44, 0.86, 0.24]); tab.axis("off")
_txt(tab, 0, 1.0, "Today's signals and target allocation", 12, "bold", ACCENT)
rows = [["Asset", "Signal on", "Ratio", "Last close", "Target", "Instrument"]]
for a in ORDER:
    v = state["assets"][a]
    rows.append([
        a.title(),
        "HOLD" if v["signal"] else "cash",
        "n/a" if v["ratio"] is None else f"{v['ratio']:.3f}",
        "n/a" if v["close"] is None else f"{v['close']:,.2f}",
        f"{state['weights'][a]*100:.1f}%"
        + (" (cap)" if (v["cap"] is not None and v["signal"]
                        and state["weights"][a] >= v["cap"] - 1e-9) else ""),
        f"{v['tradeable']} | {v['venue'].title()}",
    ])
rows.append(["Cash", "", "", "", f"{state['cash']*100:.1f}%", ""])

t = tab.table(cellText=rows[1:], colLabels=rows[0],
              cellLoc="left", colLoc="left",
              colWidths=[0.14, 0.13, 0.11, 0.17, 0.19, 0.26],
              bbox=[0, 0.05, 1, 0.85])
t.auto_set_font_size(False); t.set_fontsize(8.5)
for (r, c), cell in t.get_celld().items():
    cell.set_edgecolor("#dddddd")
    if r == 0:
        cell.set_facecolor(ACCENT)
        cell.set_text_props(color="white", weight="bold")
    elif r == len(rows) - 1:
        cell.set_facecolor("#f5f5f5")
        cell.set_text_props(style="italic")
    elif rows[r][1] == "HOLD":
        cell.set_facecolor("#eef7ee")

# Allocation bar
bar = fig.add_axes([0.08, 0.27, 0.86, 0.13])
left = 0.0
for a in ORDER:
    w = state["weights"][a]
    if w <= 0:
        continue
    bar.barh(0, w, left=left, color=PALETTE[a], edgecolor="white")
    if w > 0.06:
        bar.text(left + w / 2, 0, f"{a}\n{w*100:.0f}%", ha="center",
                 va="center", size=8.5, color="white", weight="bold")
    left += w
if state["cash"] > 0.005:
    bar.barh(0, state["cash"], left=left, color="#cccccc",
             edgecolor="white")
    if state["cash"] > 0.06:
        bar.text(left + state["cash"] / 2, 0,
                 f"CASH\n{state['cash']*100:.0f}%", ha="center",
                 va="center", size=8.5, color="#444444", weight="bold")
bar.set_xlim(0, 1); bar.set_ylim(-0.5, 0.5)
bar.set_yticks([]); bar.set_xticks([0, .25, .5, .75, 1])
bar.set_xticklabels(["0%", "25%", "50%", "75%", "100%"], size=8)
bar.set_title("Where the money should be right now", size=10,
              loc="left", color=ACCENT, weight="bold")
for s in bar.spines.values():
    s.set_visible(False)

# Notes / reconciliation
note = fig.add_axes([0.08, 0.05, 0.86, 0.19]); note.axis("off")
y = 1.0
_txt(note, 0, y, "Notes", 11, "bold", ACCENT); y -= 0.14
msgs = []
if broker:
```

```
            real = broker.get("total_value")
            if real is not None and st:
                diff = real / st["nav"] - 1
                msgs.append(f"Broker NAV ${real:,.2f} vs paper ${st['nav']:,.2f} "
                            f"({diff*100:+.2f}% tracking difference).")
            msgs.append(f"Broker snapshot taken {broker.get('asof', 'unknown')}.")
        else:
            msgs.append("PAPER MODE - no broker snapshot found. Figures below are "
                        "simulated, not realised. No money is at risk.")
        if state["errors"]:
            msgs.append(f"No price data for {', '.join(state['errors'])}; treated "
                        f"as 'cash' (fail-safe). Investigate before trading.")
        msgs.append("Paper NAV assumes execution at the daily close with no fees, "
                    "spread, slippage or tax. Live results will be lower.")
        msgs.append(f"Solana is capped at "
                    f"{ASSETS['SOL']['cap']*100:.0f}% of the portfolio by design.")
        for m in msgs:
            for line in _wrap(m, 105):
                _txt(note, 0, y, line, 8.5); y -= 0.085

        pdf.savefig(fig); plt.close(fig)

        # -- Page 2: curves ----------------------------------------
        if nav is not None and len(nav) > 1:
            fig = plt.figure(figsize=(8.27, 11.69))
            fig.subplots_adjust(left=0.10, right=0.94, top=0.94, bottom=0.06,
                                hspace=0.35)

            ax1 = fig.add_subplot(3, 1, 1)
            ax1.plot(nav.index, nav.values, color=ACCENT, lw=1.6)
            ax1.fill_between(nav.index, st["start_nav"], nav.values,
                             where=(nav.values >= st["start_nav"]),
                             color=GREEN, alpha=0.12)
            ax1.fill_between(nav.index, st["start_nav"], nav.values,
                             where=(nav.values < st["start_nav"]),
                             color=RED, alpha=0.12)
            ax1.axhline(st["start_nav"], color=GREY, ls="--", lw=0.8)
            ax1.set_title("Paper NAV since inception", size=11, loc="left",
                          color=ACCENT, weight="bold")
            ax1.set_ylabel("USD")
            ax1.grid(alpha=0.25)

            ax2 = fig.add_subplot(3, 1, 2)
            dd = (nav / nav.cummax() - 1) * 100
            ax2.fill_between(dd.index, dd.values, 0, color=RED, alpha=0.3)
            ax2.plot(dd.index, dd.values, color=RED, lw=1.0)
            ax2.set_title("Drawdown from peak", size=11, loc="left",
                          color=ACCENT, weight="bold")
            ax2.set_ylabel("%")
            ax2.grid(alpha=0.25)

            ax3 = fig.add_subplot(3, 1, 3)
            for i, a in enumerate(ORDER):
                on = W[a] > 0
                ax3.fill_between(W.index, i - 0.4, i + 0.4, where=on,
                                 color=PALETTE[a], alpha=0.85,
                                 step="post", linewidth=0)
            ax3.set_yticks(range(len(ORDER)))
            ax3.set_yticklabels([a.title() for a in ORDER], size=9)
            ax3.set_ylim(-0.6, len(ORDER) - 0.4)
            ax3.set_title("When each asset was held (shaded = in position)",
                          size=11, loc="left", color=ACCENT, weight="bold")
            ax3.grid(alpha=0.2, axis="x")

            pdf.savefig(fig); plt.close(fig)

        d = pdf.infodict()
        d["Title"] = f"Recycling Portfolio Report {week}"
        d["Author"] = "signals.py / weekly_report.py"


def _wrap(s, n):
    words, lines, cur = s.split(), [], ""
    for w in words:
        if len(cur) + len(w) + 1 > n:
            lines.append(cur); cur = w
        else:
            cur = f"{cur} {w}".strip()
    if cur:
        lines.append(cur)
    return lines


# -- Main ------------------------------------------------------------

def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--capital", type=float, default=10000.0,
                   help="Starting paper capital. Default: 10000")
    p.add_argument("--outdir", default=os.path.expanduser("~/portfolio/reports"))
    p.add_argument("--accounts", default=os.path.expanduser("~/portfolio/accounts.json"),
                   help="Optional broker snapshot written by the execution layer")
    p.add_argument("--inception", default=INCEPTION)
    args = p.parse_args()

    os.makedirs(args.outdir, exist_ok=True)

    state = build_state()
    nav, W, _ = build_paper_curve(args.capital, args.inception)
    st = stats(nav)
    if st and W is not None:
```

```
        st["deployed_pct"] = float((W.sum(axis=1) > 0).mean())

    broker = None
    if os.path.exists(args.accounts):
        try:
            with open(args.accounts) as fh:
                broker = json.load(fh)
        except Exception: # noqa: BLE001
            broker = None

    week = datetime.now(timezone.utc).strftime("%G-W%V")
    out = os.path.join(args.outdir, f"portfolio-{week}.pdf")
    render_pdf(out, state, st, nav, W, broker)
    shutil.copyfile(out, os.path.join(args.outdir, "latest.pdf"))

    if nav is not None:
        nav.rename("nav").to_csv(os.path.join(args.outdir, "nav_history.csv"),
                                 index_label="date")

    summary = {"week": week, "report": out, "stats": st,
               "signals": {a: state["assets"][a]["signal"] for a in ORDER},
               "ratios": {a: state["assets"][a]["ratio"] for a in ORDER},
               "weights": state["weights"], "cash": state["cash"],
               "mode": "live" if broker else "paper",
               "errors": state["errors"],
               "generated": state["generated"]}
    with open(os.path.join(args.outdir, "latest.json"), "w") as fh:
        json.dump(summary, fh, indent=2)

    print(f"Wrote {out}")
    if st:
        print(f" Paper NAV ${st['nav']:,.2f} "
              f"week {(st['week_return'] or 0)*100:+.2f}% "
              f"total {st['total_return']*100:+.2f}% "
              f"maxDD {st['max_dd']*100:.1f}%")
    print(f" Mode: {summary['mode']} Active: {state['n_active']}/4")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

24. cap_sweep.py

Produced the cap decision in Section 18. It refetches prices into its own `cache/` rather than trusting the stale `timing_*.csv` files, and handles the crypto/futures calendar mismatch by treating the 7-day crypto calendar as master and forward-filling futures closes.

```
"""
Choose the Solana cap for the copper-gas-bitcoin-sol recycling portfolio.

Regenerates rolling momentum for all four assets (the saved timing_*.csv
files in the parent directory are stale and inconsistent with the study),
builds the capital-recycling portfolio for a range of SOL caps, and
reports the risk/return trade-off for each.

Calendar handling: crypto trades 7 days a week, futures do not. We use
the crypto calendar as the master, forward-fill futures closes onto
non-trading days (so a held position persists over the weekend) and
assign those days zero return. Signals likewise persist.

Usage:
    python3 cap_sweep.py # full sweep
    python3 cap_sweep.py --refresh # re-download prices
"""

import argparse
import os

import numpy as np
import pandas as pd
import yfinance as yf

CACHE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "cache")

SPEC = {
    "COPPER": ("HG=F", 180),
    "NATGAS": ("NG=F", 90),
    "BTC": ("BTC-USD", 180),
    "SOL": ("SOL-USD", 180),
}
ORDER = ["COPPER", "NATGAS", "BTC", "SOL"]
START = "2021-04-19" # first date all four have a full lookback
CAPITAL = 10_000.0


def fetch(symbol, refresh=False):
    os.makedirs(CACHE, exist_ok=True)
    path = os.path.join(CACHE, symbol.replace("=", "_").replace("-", "_") + ".csv")
    if os.path.exists(path) and not refresh:
        d = pd.read_csv(path, parse_dates=["Date"]).set_index("Date")
        return d
    h = yf.Ticker(symbol).history(start="2020-01-01", interval="1d",
                                  auto_adjust=False)
    h = h.dropna(subset=["Close", "Volume"])[["Close", "Volume"]]
```

```
    h.index = pd.to_datetime(h.index, utc=True).tz_localize(None).normalize()
    h = h[~h.index.duplicated(keep="last")]
    h.index.name = "Date"
    h.to_csv(path)
    return h


def rolling_ratio(hist, lookback):
    """Bookmaker-pool momentum ratio for each day (native calendar)."""
    closes = hist["Close"].values.astype(float)
    volumes = hist["Volume"].values.astype(float)
    out = np.full(len(closes), np.nan)
    for i in range(lookback, len(closes)):
        wc = closes[i - lookback:i + 1]
        wv = volumes[i - lookback:i + 1]
        rets = np.diff(wc) / wc[:-1]
        vols = wv[1:]
        mv = vols.mean()
        if mv == 0 or not np.isfinite(mv):
            continue
        flows = (vols / mv) * rets
        pool = float(wc[0])
        mp = pool
        for f in flows:
            pool += f * mp
            mp = pool
        if np.isfinite(mp) and closes[i] != 0:
            out[i] = mp / closes[i]
    return pd.Series(out, index=hist.index)


def build_panel(refresh=False):
    """Master calendar, per-asset returns and lagged signals."""
    hist, ratio = {}, {}
    for name, (sym, lb) in SPEC.items():
        h = fetch(sym, refresh)
        hist[name] = h
        ratio[name] = rolling_ratio(h, lb)

    cal = hist["BTC"].index # 7-day crypto calendar
    cal = cal[cal >= pd.Timestamp(START)]

    rets, sigs = {}, {}
    for name in ORDER:
        close = hist[name]["Close"].reindex(cal).ffill()
        rets[name] = close.pct_change().fillna(0.0)
        s = (ratio[name] > 1.0).where(ratio[name].notna())
        s = s.reindex(cal).ffill().fillna(0.0).astype(bool)
        sigs[name] = s.shift(1).fillna(False).astype(bool) # trade on yesterday

    return pd.DataFrame(rets)[ORDER], pd.DataFrame(sigs)[ORDER]


def weights_for(sig_row, cap):
    """Equal weight across active assets; SOL capped, excess to others."""
    active = [a for a in ORDER if sig_row[a]]
    w = {a: 0.0 for a in ORDER}
    if not active:
        return w
    for a in active:
        w[a] = 1.0 / len(active)
    if cap is not None and w["SOL"] > cap:
        excess = w["SOL"] - cap
        w["SOL"] = cap
        others = [a for a in active if a != "SOL"]
        if others:
            for a in others:
                w[a] += excess / len(others)
    return w


def run(P, S, cap):
    W = pd.DataFrame(
        [weights_for(S.loc[d], cap) for d in S.index],
        index=S.index, columns=ORDER)
    port = (W * P).sum(axis=1)
    nav = CAPITAL * (1 + port).cumprod()
    eq = nav / CAPITAL
    dd = nav / nav.cummax() - 1
    years = (nav.index[-1] - nav.index[0]).days / 365.25
    r = port
    downside = r[r < 0].std()
    return {
        "cap": cap,
        "final": nav.iloc[-1],
        "total": eq.iloc[-1] - 1,
        "cagr": eq.iloc[-1] ** (1 / years) - 1,
        "vol": r.std() * np.sqrt(365),
        "sharpe": r.mean() / r.std() * np.sqrt(365) if r.std() > 0 else np.nan,
        "sortino": (r.mean() / downside * np.sqrt(365)
                    if downside and downside > 0 else np.nan),
        "maxdd": dd.min(),
        "calmar": ((eq.iloc[-1] ** (1 / years) - 1) / abs(dd.min())
                   if dd.min() < 0 else np.nan),
        "sol_avg_w": W["SOL"].mean(),
        "sol_share_of_risk": None,
        "worst_day": r.min(),
        "nav": nav,
        "W": W,
    }
```

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--refresh", action="store_true")
    args = ap.parse_args()

    P, S = build_panel(args.refresh)

    print(f"Period: {S.index[0].date()} to {S.index[-1].date()} "
          f"({len(S)} days, {(S.index[-1]-S.index[0]).days/365.25:.2f} years)")
    print(f"Start capital: ${CAPITAL:,.0f}\n")

    print("Signal activity (fraction of days each asset says HOLD):")
    for a in ORDER:
        print(f" {a:7s} {S[a].mean()*100:5.1f}%")
    n_active = S.sum(axis=1)
    print("\nHow many assets active at once:")
    for k in range(5):
        print(f" {k} active: {(n_active==k).mean()*100:5.1f}%")
    print(f" capital deployed: {(n_active>0).mean()*100:.1f}% of days")
    print("\nOverlap: fraction of SOL-active days where SOL is the ONLY signal:")
    solonly = ((S["SOL"]) & (n_active == 1)).sum() / max(1, S["SOL"].sum())
    print(f" {solonly*100:.1f}% <- these are the days an uncapped rule "
          f"puts 100% in SOL")

    caps = [0.0, 0.10, 0.15, 0.20, 0.25, 0.3333, 0.50, None]
    results = [run(P, S, c) for c in caps]

    print("\n" + "=" * 104)
    print(f"{'Cap':>7s} {'Final $':>12s} {'CAGR':>8s} {'Vol':>7s} "
          f"{'Sharpe':>7s} {'Sortino':>8s} {'MaxDD':>8s} {'Calmar':>7s} "
          f"{'AvgSOLw':>8s} {'WorstDay':>9s}")
    print("=" * 104)
    for r in results:
        lab = "none" if r["cap"] is None else f"{r['cap']*100:.0f}%"
        print(f"{lab:>7s} {r['final']:>12,.0f} {r['cagr']*100:>7.1f}% "
              f"{r['vol']*100:>6.1f}% {r['sharpe']:>7.2f} {r['sortino']:>8.2f} "
              f"{r['maxdd']*100:>7.1f}% {r['calmar']:>7.2f} "
              f"{r['sol_avg_w']*100:>7.1f}% {r['worst_day']*100:>8.1f}%")
    print("=" * 104)

    # Marginal analysis: what each step up in the cap buys and costs.
    print("\nMarginal trade-off, relative to the uncapped rule:")
    base = [r for r in results if r["cap"] is None][0]
    print(f"{'Cap':>7s} {'% of uncapped return kept':>26s} "
          f"{'% of uncapped drawdown taken':>29s} {'return kept per unit risk':>26s}")
    for r in results:
        if r["cap"] is None or r["cap"] == 0.0:
            continue
        kept = r["total"] / base["total"]
        pain = r["maxdd"] / base["maxdd"]
        print(f"{r['cap']*100:>6.0f}% {kept*100:>25.1f}% {pain*100:>28.1f}% "
              f"{kept/pain:>25.2f}")

    os.makedirs(CACHE, exist_ok=True)
    out = pd.DataFrame([{k: v for k, v in r.items()
                         if k not in ("nav", "W")} for r in results])
    out.to_csv(os.path.join(CACHE, "cap_sweep.csv"), index=False)
    pd.DataFrame({("none" if r["cap"] is None else f"{r['cap']:.4f}"): r["nav"]
                  for r in results}).to_csv(os.path.join(CACHE, "cap_navs.csv"))
    print(f"\nWrote {CACHE}/cap_sweep.csv and cap_navs.csv")


if __name__ == "__main__":
    main()
```

25. run_weekly.sh

The cron entry point. Absolute paths throughout, because cron's `PATH` is nearly empty and it does not read a shell profile.

```
#!/usr/bin/env bash
# Generate this week's report. Runs on the OCI instance, under cron.
set -euo pipefail
cd /home/ubuntu/portfolio
mkdir -p reports logs
echo "=== $(date -u +%Y-%m-%dT%H:%M:%SZ) starting weekly report ==="
./venv/bin/python weekly_report.py \
    --capital "${PORTFOLIO_CAPITAL:-10000}" \
    --outdir /home/ubuntu/portfolio/reports \
    --accounts /home/ubuntu/portfolio/accounts.json
echo "=== $(date -u +%Y-%m-%dT%H:%M:%SZ) done ==="
# Keep the last 52 weekly reports; drop older ones.
ls -1t /home/ubuntu/portfolio/reports/portfolio-*.pdf 2>/dev/null \
    | tail -n +53 | xargs -r rm -f
```

26. keepalive.sh

Stops Oracle reclaiming the instance, and writes the heartbeat used by `healthcheck.sh`. The heartbeat is written *before* the CPU burn so the timestamp survives even if the burn is killed.

```
#!/usr/bin/env bash
# Keep an Oracle Always Free instance from being reclaimed as "idle".
#
# Oracle may reclaim an Always Free compute instance when, across a
# 7-day window, ALL of the following are true:
# - CPU 95th percentile < 20%
# - Network 95th percentile < 20%
# - Memory 95th percentile < 20% (Ampere A1 shapes only)
#
# Because the test is a conjunction, defeating ONE criterion is enough.
# CPU is the cheapest to move, so that is what we do: burn one core at
# the lowest possible priority for a few minutes each half hour. That
# puts well over 5% of samples above the threshold, which is what the
# 95th percentile needs.
#
# 'nice -n 19' plus 'ionice -c3' means this yields to absolutely
# everything else on the box -- the AI agents and the Gateway app never
# wait on it.
#
# Cron: */30 * * * * /home/ubuntu/portfolio/keepalive.sh
set -euo pipefail

BURN_SECONDS="${KEEPALIVE_BURN_SECONDS:-300}"
STATE_DIR="${KEEPALIVE_STATE_DIR:-/home/ubuntu/portfolio}"
HEARTBEAT="$STATE_DIR/heartbeat.txt"

mkdir -p "$STATE_DIR"

# Heartbeat first, so the timestamp is written even if the burn is killed.
{
    echo "last_keepalive=$(date -u +%Y-%m-%dT%H:%M:%SZ)"
    echo "uptime=$(uptime -p 2>/dev/null || true)"
    echo "load=$(cut -d' ' -f1-3 /proc/loadavg)"
    echo "burn_seconds=$BURN_SECONDS"
} > "$HEARTBEAT"

# One core, lowest priority, hard timeout. Prefer stress-ng if present.
if command -v stress-ng >/dev/null 2>&1; then
    nice -n 19 ionice -c3 stress-ng --cpu 1 --timeout "${BURN_SECONDS}s" \
        --quiet >/dev/null 2>&1 || true
else
    # Portable fallback: a busy loop that cannot outlive the timeout.
    nice -n 19 ionice -c3 timeout "${BURN_SECONDS}" \
        bash -c 'while :; do :; done' >/dev/null 2>&1 || true
fi

echo "burn_completed=$(date -u +%Y-%m-%dT%H:%M:%SZ)" >> "$HEARTBEAT"
```

27. healthcheck.sh

Answers “is it alive and on schedule?” and exits non-zero if not.

```
#!/usr/bin/env bash
# Is the portfolio reporter alive and on schedule?
#
# Run on the server, or remotely: ssh oci '/home/ubuntu/portfolio/healthcheck.sh'
# Exit code 0 = healthy, 1 = something needs attention.
set -uo pipefail

DIR="${PORTFOLIO_DIR:-/home/ubuntu/portfolio}"
REPORTS="$DIR/reports"
NOW=$(date -u +%s)
PROBLEMS=0

say() { printf ' %-22s %s\n' "$1" "$2"; }
fail() { printf ' %-22s %s\n' "$1" "$2"; PROBLEMS=$((PROBLEMS+1)); }

echo "Portfolio health $(date -u +%Y-%m-%dT%H:%M:%SZ)"
echo

# --- keepalive ---
if [ -f "$DIR/heartbeat.txt" ]; then
    HB=$(grep -m1 '^last_keepalive=' "$DIR/heartbeat.txt" | cut -d= -f2)
    HB_S=$(date -u -d "$HB" +%s 2>/dev/null || echo 0)
    AGE=$(( (NOW - HB_S) / 60 ))
    if [ "$HB_S" -gt 0 ] && [ "$AGE" -lt 90 ]; then
        say "keepalive" "ok - ${AGE} min ago"
    else
        fail "keepalive" "STALE - last seen $HB (${AGE} min ago)"
    fi
else
    fail "keepalive" "MISSING - $DIR/heartbeat.txt not found"
fi

# --- latest report ---
if [ -f "$REPORTS/latest.pdf" ]; then
    R_S=$(stat -c %Y "$REPORTS/latest.pdf" 2>/dev/null || echo 0)
    DAYS=$(( (NOW - R_S) / 86400 ))
    if [ "$DAYS" -le 8 ]; then
        say "weekly report" "ok - ${DAYS} day(s) old"
    else
        fail "weekly report" "OVERDUE - ${DAYS} days old"
    fi
    say "report count" "$(ls -1 "$REPORTS"/portfolio-*.pdf 2>/dev/null | wc -l | tr -d ' ') archived"
else
    fail "weekly report" "MISSING - no latest.pdf; has it ever run?"
fi
```

```
# --- signal freshness / errors ---
if [ -f "$REPORTS/latest.json" ]; then
    ERRS=$(python3 -c "
import json;d=json.load(open('$REPORTS/latest.json'))
e=d.get('errors') or []
print(('errors: '+','.join(e)) if e else 'no data errors')
print('mode: '+d.get('mode','?'))" 2>/dev/null)
    while IFS= read -r line; do
        case "$line" in
            errors:*) fail "signals" "$line" ;;
            *) say "signals" "$line" ;;
        esac
    done <<< "$ERRS"
fi

# --- cron ---
if crontab -l 2>/dev/null | grep -q 'run_weekly.sh'; then
    say "cron: report" "installed"
else
    fail "cron: report" "NOT INSTALLED"
fi
if crontab -l 2>/dev/null | grep -q 'keepalive.sh'; then
    say "cron: keepalive" "installed"
else
    fail "cron: keepalive" "NOT INSTALLED"
fi

# --- disk (the only thing that costs money on the free tier) ---
say "disk" "$(df -h / | awk 'NR==2{print $4" free of "$2" ("$5" used)"}')"

echo
if [ "$PROBLEMS" -eq 0 ]; then
    echo "HEALTHY"
else
    echo "$PROBLEMS problem(s) need attention"
fi
exit $(( PROBLEMS > 0 ))
```

28. install_oci.sh

Idempotent deploy from the Mac. Re-running it is the normal way to ship a change. It runs one report immediately, so a broken install is discovered now rather than next Monday.

```
#!/usr/bin/env bash
# Install the recycling-portfolio reporter on the OCI instance.
#
# Run this FROM YOUR MAC. It copies the scripts to the server, builds a
# virtualenv, runs one report to prove the install works, and installs a
# weekly cron job. Idempotent: safe to run repeatedly.
#
# bash install_oci.sh
#
# Assumes an ssh alias 'oci' exists (see oci.pdf section on SSH).

set -euo pipefail

HOST="${OCI_HOST:-oci}"
REMOTE_DIR="/home/ubuntu/portfolio"
LOCAL_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

echo "==> Checking SSH access to '$HOST'"
ssh -o ConnectTimeout=15 -o BatchMode=yes "$HOST" 'echo " connected as $(whoami)@$(hostname)"'

echo "==> Creating $REMOTE_DIR"
ssh "$HOST" "mkdir -p $REMOTE_DIR/reports $REMOTE_DIR/logs"

echo "==> Copying scripts"
scp -q "$LOCAL_DIR/signals.py" "$LOCAL_DIR/weekly_report.py" \
      "$LOCAL_DIR/cap_sweep.py" "$LOCAL_DIR/run_weekly.sh" \
      "$LOCAL_DIR/keepalive.sh" "$LOCAL_DIR/healthcheck.sh" \
      "$HOST:$REMOTE_DIR/"
ssh "$HOST" "chmod +x $REMOTE_DIR/run_weekly.sh $REMOTE_DIR/keepalive.sh $REMOTE_DIR/healthcheck.sh"

echo "==> Building virtualenv (aarch64 wheels; this takes a few minutes)"
ssh "$HOST" "bash -s" <<'REMOTE'
set -euo pipefail
cd /home/ubuntu/portfolio
sudo apt-get update -qq
sudo apt-get install -y -qq python3-venv python3-dev >/dev/null
if [ ! -d venv ]; then python3 -m venv venv; fi
./venv/bin/pip install --quiet --upgrade pip wheel
./venv/bin/pip install --quiet numpy pandas matplotlib yfinance
echo " installed: $(./venv/bin/python -c 'import numpy,pandas,matplotlib,yfinance as y; print("numpy",numpy.__version__,"pandas",pandas.
      __version__,"yfinance",y.__version__)')"
REMOTE

echo "==> Running one report now (proves the install works)"
ssh "$HOST" "cd $REMOTE_DIR && ./run_weekly.sh"

echo "==> Starting keepalive once, to prove it runs"
ssh "$HOST" "KEEPALIVE_BURN_SECONDS=5 $REMOTE_DIR/keepalive.sh && cat $REMOTE_DIR/heartbeat.txt"

echo "==> Installing cron jobs (weekly report + Oracle idle keepalive)"
ssh "$HOST" "bash -s" <<'REMOTE'
set -euo pipefail
```

```
REPORT='15 21 * * 0 /home/ubuntu/portfolio/run_weekly.sh >> /home/ubuntu/portfolio/logs/cron.log 2>&1'
ALIVE='*/30 * * * * /home/ubuntu/portfolio/keepalive.sh >> /home/ubuntu/portfolio/logs/keepalive.log 2>&1'
TMP=$(mktemp)
crontab -l 2>/dev/null \
  | grep -v 'portfolio/run_weekly.sh' \
  | grep -v 'portfolio/keepalive.sh' > "$TMP" || true
echo '# Recycling portfolio: weekly report + keep instance from being reclaimed' >> "$TMP"
echo "$REPORT" >> "$TMP"
echo "$ALIVE" >> "$TMP"
crontab "$TMP"
rm -f "$TMP"
echo " cron installed:"
crontab -l | grep -A2 'Recycling portfolio'
REMOTE

echo "==> Health check"
ssh "$HOST" "$REMOTE_DIR/healthcheck.sh" || true

echo
echo "==> Done. Fetch the latest report with:"
echo " bash $LOCAL_DIR/fetch_report.sh"
```

29. fetch_report.sh

Pulls the latest report back to the Mac and opens it in Skim.

```
#!/usr/bin/env bash
# Pull the latest report off the OCI instance and open it.
# bash fetch_report.sh # latest
# bash fetch_report.sh 2026-W35 # a specific week
set -euo pipefail
HOST="${OCI_HOST:-oci}"
DEST="${REPORT_DEST:-$HOME/git/andy/crypto/portfolio/reports}"
mkdir -p "$DEST"

if [ $# -ge 1 ]; then REMOTE="portfolio-$1.pdf"; else REMOTE="latest.pdf"; fi

scp -q "$HOST:/home/ubuntu/portfolio/reports/$REMOTE" "$DEST/$REMOTE"
scp -q "$HOST:/home/ubuntu/portfolio/reports/latest.json" "$DEST/latest.json" 2>/dev/null || true

echo "Fetched $DEST/$REMOTE"
python3 -c "
import json,sys
try: d=json.load(open('$DEST/latest.json'))
except Exception: sys.exit()
s=d.get('stats') or {}
print(f\" week {d['week']} mode={d['mode']}\")
if s: print(f\" NAV \${s['nav']:,.2f} week {(s.get('week_return') or 0)*100:+.2f}% total {s['total_return']*100:+.2f}% maxDD {s['max_dd']*100:.1f
      }%\")
print(' weights: ' + ', '.join(f\"{k} {v*100:.0f}%\" for k,v in d['weights'].items() if v>0) + f\", cash {d['cash']*100:.0f}%\")
"
open -a Skim "$DEST/$REMOTE" 2>/dev/null || true
```

30. make_figure.py

Self-contained: fetches prices from Yahoo Finance and draws the three-way portfolio comparison with no dependency on any other file here. Presented with a walkthrough in `algorithm.pdf`.

```
"""
Generate the copper-gas-bitcoin-sol comparison diagram, from scratch.

Self-contained: fetches prices from yfinance, computes the bookmaker-pool
momentum signal for each asset, applies the capital-recycling rule with
the 25% Solana cap, and plots the result against an index fund.

    python3 make_figure.py # writes three_way.png
    python3 make_figure.py --cap 0.20 # try a different Solana cap
    python3 make_figure.py --capital 25000

Requires: numpy pandas matplotlib yfinance
"""

import argparse
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# -- What we trade, and how the signal is computed for each -----------
# lookback is the per-asset optimum from the cross-asset study:
# 180 days for everything except natural gas, where 90 works better.

ASSETS = {
    "COPPER": {"symbol": "HG=F", "lookback": 180},
    "NATGAS": {"symbol": "NG=F", "lookback": 90},
    "BTC": {"symbol": "BTC-USD", "lookback": 180},
    "SOL": {"symbol": "SOL-USD", "lookback": 180},
}
ORDER = ["COPPER", "NATGAS", "BTC", "SOL"]
```

```
BENCHMARK = "^SP500TR" # S&P 500 TOTAL return: dividends reinvested.
                       # The honest benchmark - a price-only index
                       # would understate it by ~2% a year.

START = "2021-04-19" # first date on which all four have a full
                       # lookback window behind them
CAPITAL = 10_000.0
SOL_CAP = 0.25

ACCENT, GREEN, GREY, PURPLE = "#1e4678", "#287828", "#8a94a0", "#7a3fb8"


# -- 1. Prices ------------------------------------------------------

def fetch(symbol, start="2020-01-01"):
    """Daily closes and volumes, tz-stripped and de-duplicated."""
    h = yf.Ticker(symbol).history(start=start, interval="1d",
                                  auto_adjust=False)
    h = h.dropna(subset=["Close"])
    h.index = pd.to_datetime(h.index, utc=True).tz_localize(None).normalize()
    return h[~h.index.duplicated(keep="last")]


# -- 2. The momentum signal (the "bookmaker pool") ------------------

def rolling_ratio(hist, lookback):
    """
    Momentum ratio for every day with a full lookback window.

    Each day's net flow is (volume / mean volume) * return: a price move
    counts for more when more money stood behind it. Those flows are
    compounded into a pool seeded at the price at the start of the
    window. The pool's final value is the 'momentum price'; we return
    its ratio to the actual close.

        ratio > 1 -> the weight of money supports a higher price
        ratio <= 1 -> money is leaving; stand aside
    """
    closes = hist["Close"].values.astype(float)
    volumes = hist["Volume"].values.astype(float)
    out = np.full(len(closes), np.nan)

    for i in range(lookback, len(closes)):
        window_close = closes[i - lookback:i + 1]
        window_vol = volumes[i - lookback:i + 1]

        returns = np.diff(window_close) / window_close[:-1]
        vols = window_vol[1:]

        mean_vol = vols.mean()
        if mean_vol == 0 or not np.isfinite(mean_vol):
            continue
        net_flows = (vols / mean_vol) * returns

        pool = float(window_close[0])
        momentum_price = pool
        for flow in net_flows:
            pool += flow * momentum_price
            momentum_price = pool

        if np.isfinite(momentum_price) and closes[i] != 0:
            out[i] = momentum_price / closes[i]

    return pd.Series(out, index=hist.index)


# -- 3. Capital recycling, with the Solana cap ----------------------

def weights_for(active, cap):
    """
    Equal weight across every asset whose signal is on, except that
    Solana is never allowed more than `cap`. The excess goes to the
    other active assets, or stays in cash if there are none.

    active: {asset: bool}. Returns {asset: weight}; the shortfall
    below 1.0 is cash.
    """
    on = [a for a in ORDER if active[a]]
    w = {a: 0.0 for a in ORDER}
    if not on:
        return w # everything in cash

    for a in on:
        w[a] = 1.0 / len(on)

    if cap is not None and w["SOL"] > cap:
        excess = w["SOL"] - cap
        w["SOL"] = cap
        others = [a for a in on if a != "SOL"]
        for a in others: # empty => stays cash
            w[a] += excess / len(others)

    return w


# -- 4. Build the daily panel ---------------------------------------

def build_panel(start=START):
    """
    Returns (returns, signals), both indexed by the 7-day crypto
```

```
    calendar.

    Crypto trades every day; futures do not. We take the crypto
    calendar as master and forward-fill futures closes onto non-trading
    days, so a held position simply persists over the weekend with zero
    return. Signals persist the same way.
    """
    hist, ratio = {}, {}
    for name in ORDER:
        h = fetch(ASSETS[name]["symbol"])
        hist[name] = h
        ratio[name] = rolling_ratio(h, ASSETS[name]["lookback"])

    calendar = hist["BTC"].index
    calendar = calendar[calendar >= pd.Timestamp(start)]

    returns, signals = {}, {}
    for name in ORDER:
        close = hist[name]["Close"].reindex(calendar).ffill()
        returns[name] = close.pct_change().fillna(0.0)

        s = (ratio[name] > 1.0).where(ratio[name].notna())
        s = s.reindex(calendar).ffill().fillna(0.0).astype(bool)
        signals[name] = s.shift(1).fillna(False).astype(bool)
        # shift(1): you cannot trade on a close you have not seen yet

    return pd.DataFrame(returns)[ORDER], pd.DataFrame(signals)[ORDER]


# -- 5. Run the portfolio ---------------------------------------------

def run(returns, signals, cap, capital=CAPITAL):
    """Daily NAV for one choice of Solana cap. cap=0 excludes Solana."""
    W = pd.DataFrame([weights_for(signals.loc[d], cap) for d in signals.index],
                     index=signals.index, columns=ORDER)
    daily = (W * returns).sum(axis=1)
    return capital * (1 + daily).cumprod(), W


def summarise(label, nav, capital=CAPITAL):
    years = (nav.index[-1] - nav.index[0]).days / 365.25
    growth = nav.iloc[-1] / capital
    drawdown = (nav / nav.cummax() - 1).min()
    print(f" {label:34s} ${nav.iloc[-1]:>10,.0f} "
          f"{(growth - 1) * 100:>+8.0f}% "
          f"CAGR {(growth ** (1 / years) - 1) * 100:>5.1f}% "
          f"worst drop {drawdown * 100:>6.1f}%")


# -- 6. The diagram ---------------------------------------------------

def main():
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--cap", type=float, default=SOL_CAP,
                    help="Solana cap as a fraction. Default: 0.25")
    ap.add_argument("--capital", type=float, default=CAPITAL)
    ap.add_argument("--out", default="three_way.png")
    args = ap.parse_args()

    print("Fetching prices and computing momentum (this takes a minute)...")
    returns, signals = build_panel()

    nav_three, _ = run(returns, signals, cap=0.0, capital=args.capital)
    nav_four, _ = run(returns, signals, cap=args.cap, capital=args.capital)
    start, end = nav_three.index[0], nav_three.index[-1]

    # Benchmark, rebased to the same starting capital on the same day.
    bench = fetch(BENCHMARK, start=str(start.date()))["Close"]
    bench = bench.reindex(nav_three.index).ffill().bfill()
    nav_index = args.capital * bench / bench.iloc[0]

    print(f"\n{start.date()} to {end.date()} "
          f"(${args.capital:,.0f} invested)\n")
    summarise("Index fund (S&P 500 total return)", nav_index, args.capital)
    summarise("Copper + Gas + Bitcoin", nav_three, args.capital)
    summarise(f"+ Sol, capped at {args.cap * 100:.0f}%", nav_four, args.capital)

    fig, ax = plt.subplots(figsize=(10.6, 5.0))

    series = [
        (nav_index, GREY, "Index fund (S&P 500, dividends reinvested)", 1.9),
        (nav_three, GREEN, "Copper + Gas + Bitcoin", 2.0),
        (nav_four, PURPLE,
         f"Copper + Gas + Bitcoin + Sol (Sol capped {args.cap * 100:.0f}%)", 2.4),
    ]
    for nav, colour, label, width in series:
        ax.plot(nav.index, nav.values, color=colour, lw=width, label=label, zorder=3)
        ax.plot(nav.index[-1], nav.iloc[-1], "o", color=colour, ms=8, zorder=4)
        ax.annotate(f"${nav.iloc[-1]:,.0f}", xy=(nav.index[-1], nav.iloc[-1]),
                    xytext=(12, 0), textcoords="offset points", color=colour,
                    fontsize=12, fontweight="bold", va="center")

    ax.axhline(args.capital, color="#c8ced6", ls="--", lw=1.1, zorder=1)
    ax.set_yscale("log")
    ax.set_yticks([10000, 25000, 50000, 100000, 200000])
    ax.set_yticklabels(["$10k", "$25k", "$50k", "$100k", "$200k"])
    ax.set_xlim(start, end + (end - start) * 0.10)
    ax.set_ylabel(f"value of a ${args.capital:,.0f} investment", fontsize=11)
    ax.grid(alpha=0.25, which="both")
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)
```

```
    ax.legend(loc="upper left", fontsize=10, framealpha=0.95)
    ax.set_title(f"${args.capital:,.0f} invested, "
                 f"{start:%B %Y} to {end:%B %Y}",
                 color=ACCENT, fontsize=13, fontweight="bold", pad=12)

    fig.tight_layout()
    fig.savefig(args.out, dpi=170, bbox_inches="tight", facecolor="white")
    print(f"\nWrote {args.out}")


if __name__ == "__main__":
    main()
```